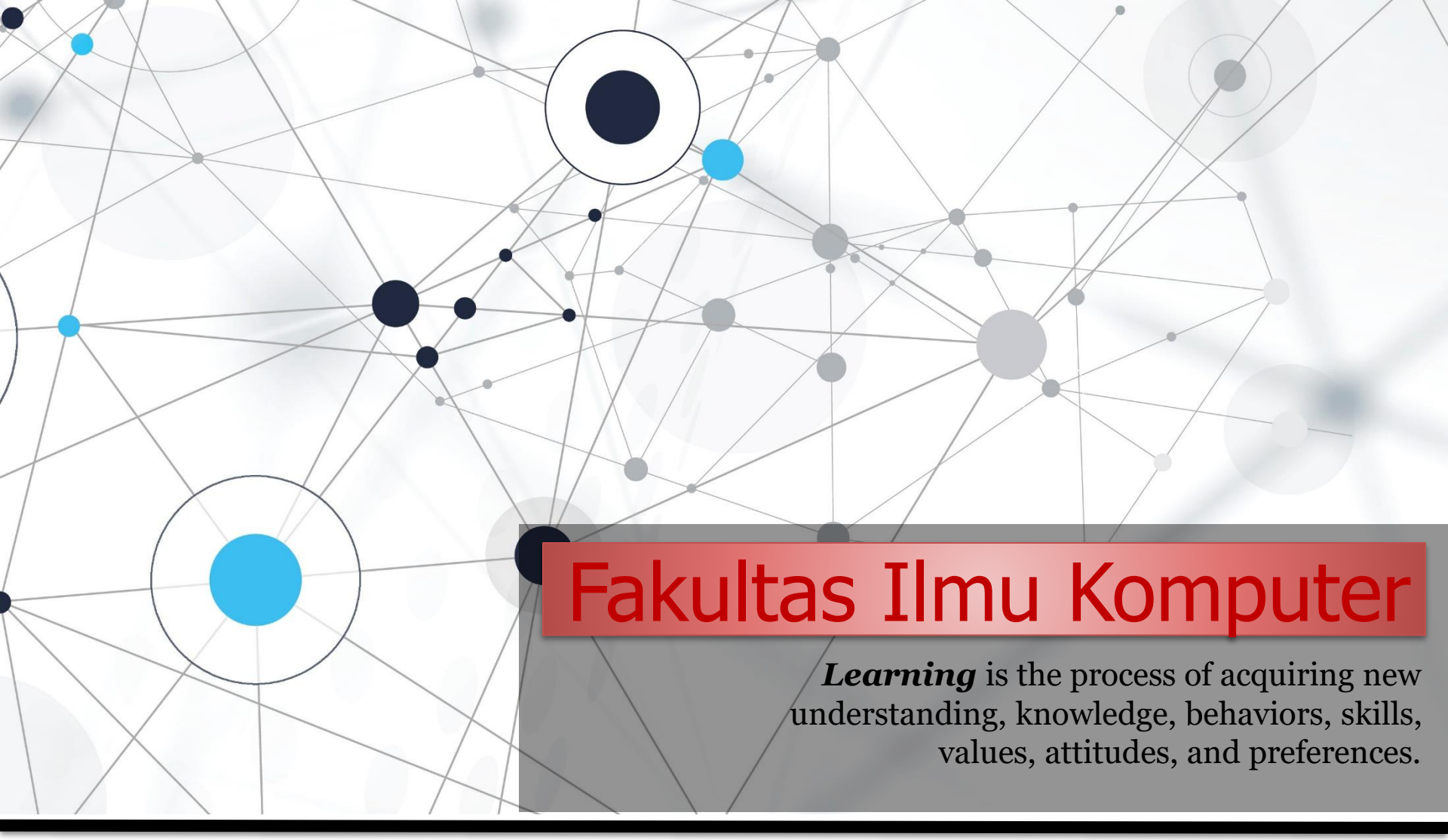


Client Side Programming Languages

(HTML, CSS & JavaScript)

Welcome to *the* Full Stack Web Development *course*

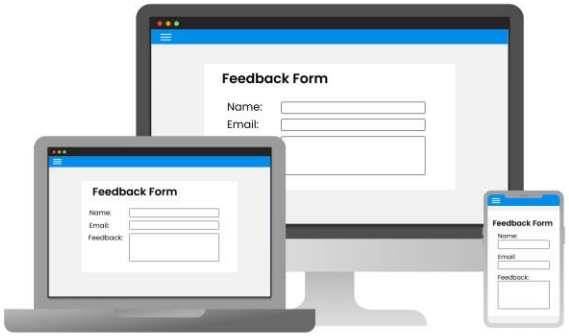
What *is* the "Client Side Programming Language"?

Client Side Programming Languages (bahasa pemrograman sisi klien) adalah bahasa pemrograman yang digunakan terutama untuk menciptakan *elemen interaktif* dan *user interface* pada sisi klien dari sebuah aplikasi desktop/mobile/*web*. Dalam pengembangan aplikasi web, sisi klien (*client side*) akan diarahkan pada context *web browser* pengguna (*user*), di mana aplikasi berjalan dan berinteraksi dengan *end-users*.

JavaScript can be used in both client-side programming and server-side programming. This is one of JavaScript's strengths, making it highly versatile/serbaguna.

Client

Client-side code runs on the user's browser and is untrusted.



Responsible for:

- ✔ Displaying the UI
- ✔ Interactivity

Server

Server-side code runs on a server and is unseen by users.

Responsible for:

- ✔ Controlling central resources
- ✔ Permissions checks
- ✔ Handling secret information

JavaScript can be used for server-side programming. This means that JavaScript code is executed on the server side to generate responses that are sent back to the client. Node.js is a popular platform for running JavaScript on the server side. In a server-side programming environment, JavaScript is used to manage business logic, access databases, handle HTTP requests, and build APIs.

1 - Client-side Development

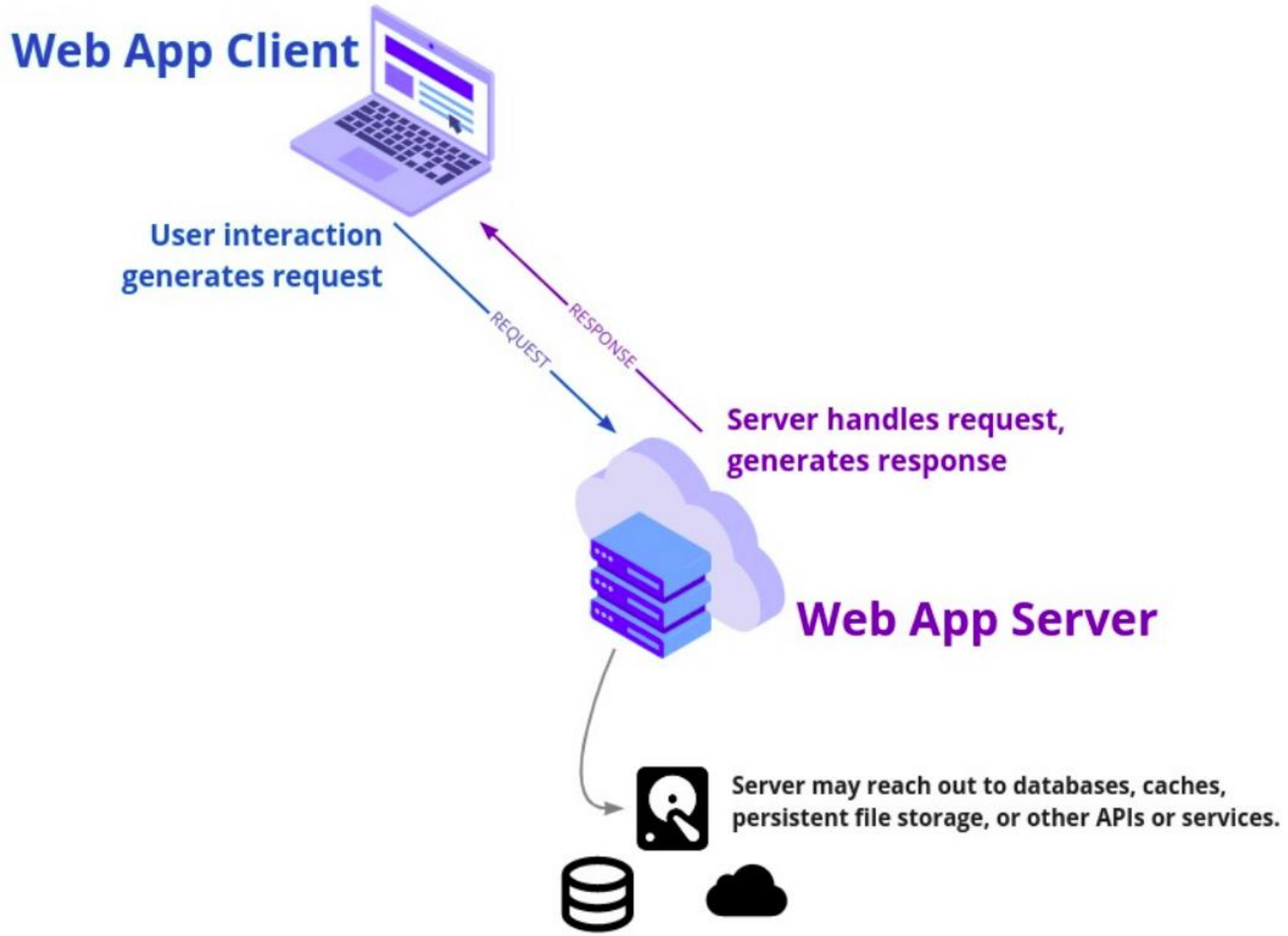
What *is* Client-side Development?

Client-side development, sometimes referred to as **front-end development**, is a type of development that involves programs that run on a client's or user's device.

Client-side developers focus on creating the part of a website with which the **user interacts**.

Client-side development focuses on the **front part** of an application that *users* can see (layouts, user interfaces, performance of websites, etc.).

Client-side Development



Client-side Programming

It is the program that runs on the client machine (***browser***) and deals with the **user interface/display and any other processing** that can happen on client machine like reading/writing cookies.

1) Interact with temporary storage

2) Make interactive web pages

3) Interact with local storage

4) Sending request for data to server

5) Send request to server

6) work as an interface between server and user

Client-side Programming

The Programming languages for client-side programming are :

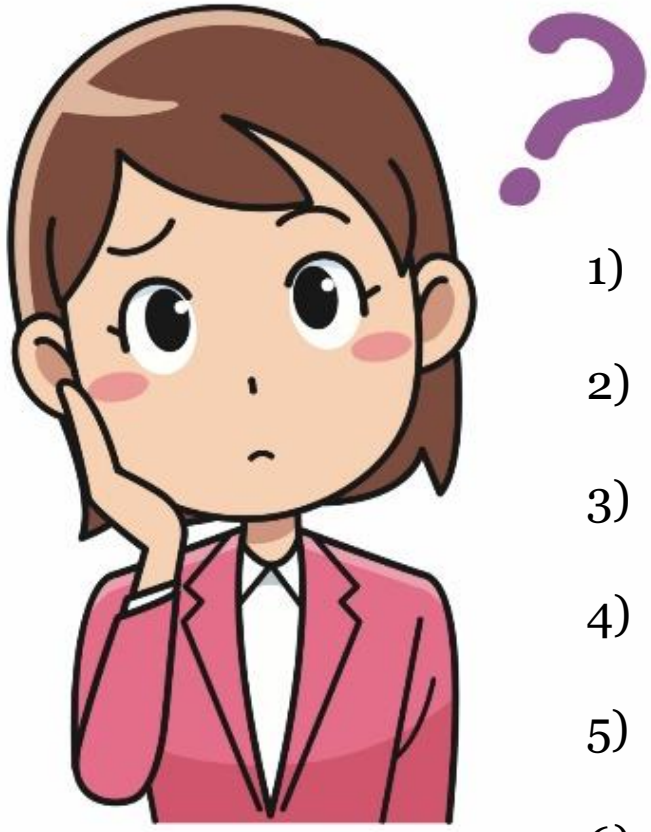
1) **HTML**, which stands for hypertext markup language, is the standard language for web development. HTML builds a website's structure and renders a website in a browser.

2) **CSS**, which stands for Cascading Style Sheets, is a design language that developers use to add visual design elements to a website coded in HTML. Developers can use CSS to make their websites look more visually appealing on users' devices.

3) **JavaScript** is a scripting language that developers can use for web development, web applications and other purposes. Developers can use JavaScript to make websites dynamic and interactive.

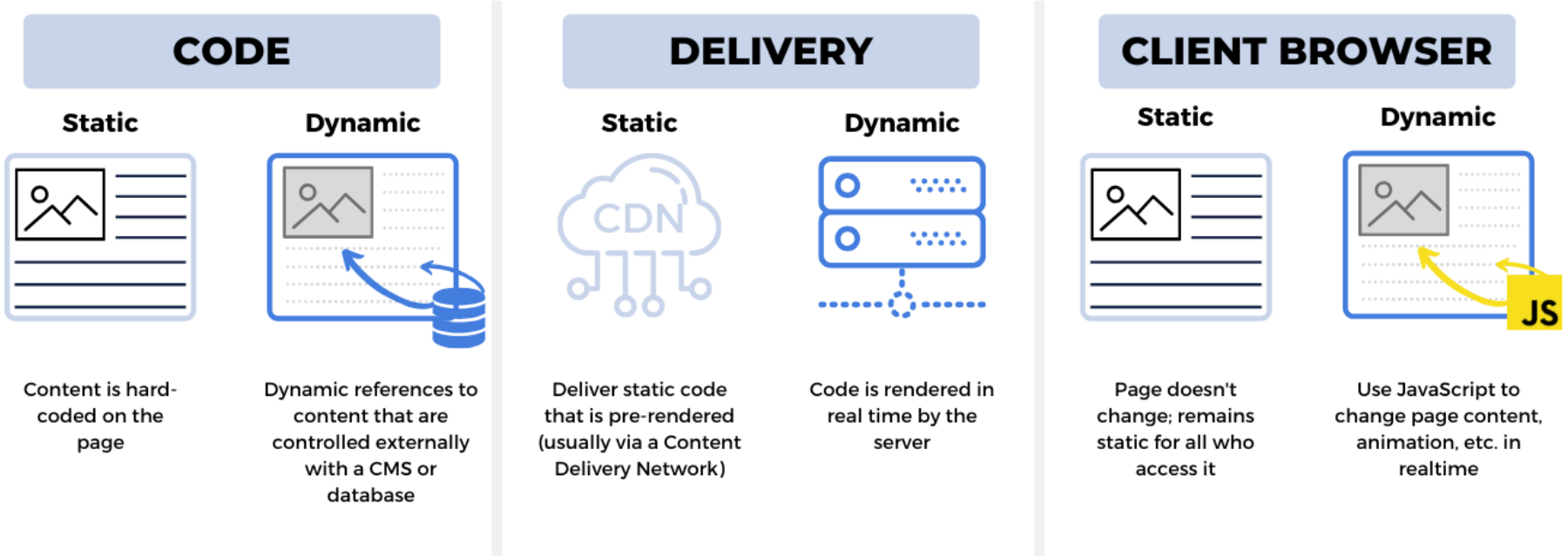
4) **VBScript** is a client-side scripting language that certain browsers support. Developers can use VBScript to add interactive elements to websites. VBScript was mainly used to give functionality to webpages. It was originally used for client-side scripting in IE. Web developers could write and embed executable VBScript functions in the Hypertext Markup Language (HTML) of a webpage, which controls the presentation of data.

Common Client-side Tasks



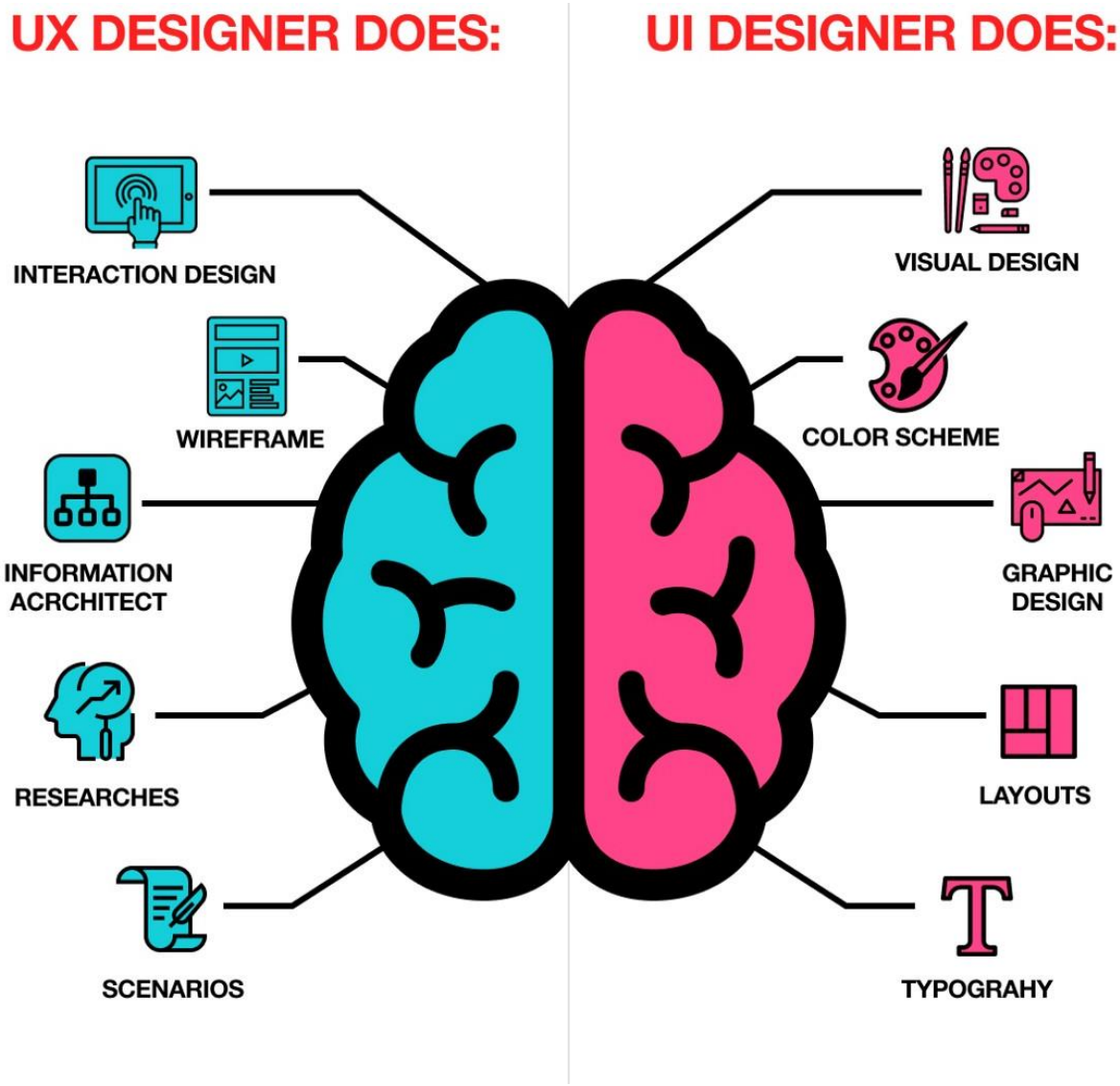
1) Creating website layouts.
2) Designing user interfaces.
3) Adding form validation.
4) Reviewing the performance of websites.
5) Adding visual design elements like colors and fonts.
6) Making website features more functional.
7) Resolving any issues that users encounter on a site.

Creating Website Layouts



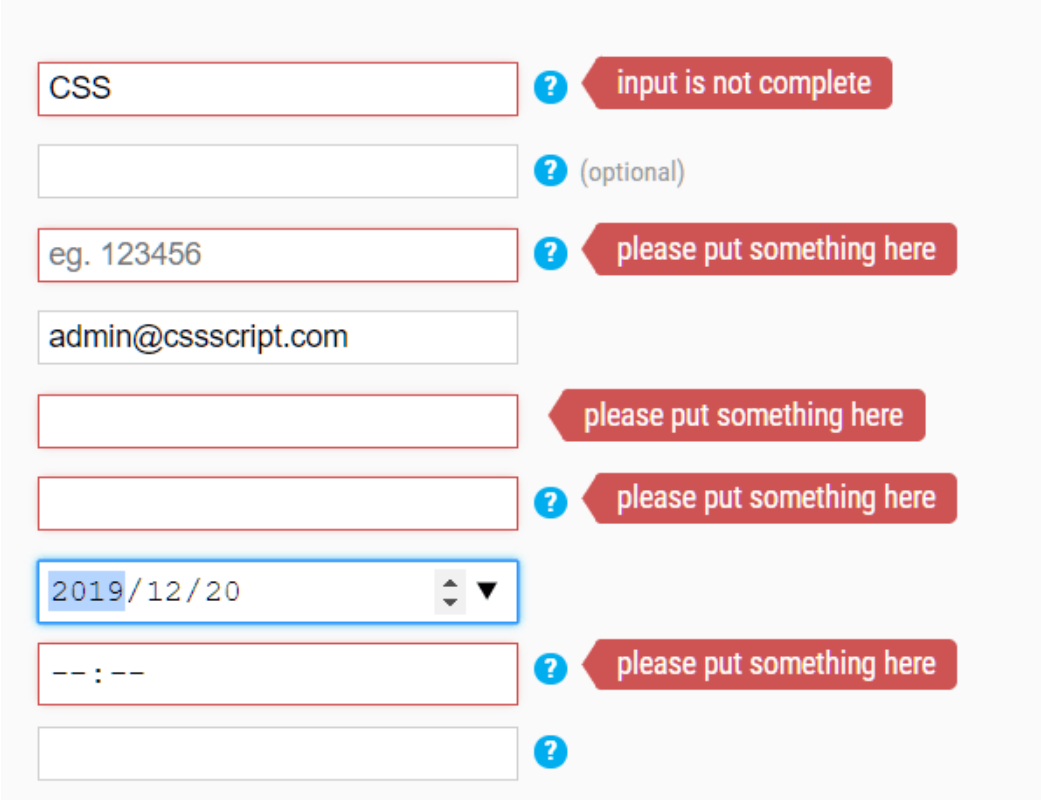
A **dynamic website** is a website that displays different types of content every time a user views it. This display changes depending on a number of factors like viewer demographics, time of day, location, language settings, *etc*.

Designing User Interfaces



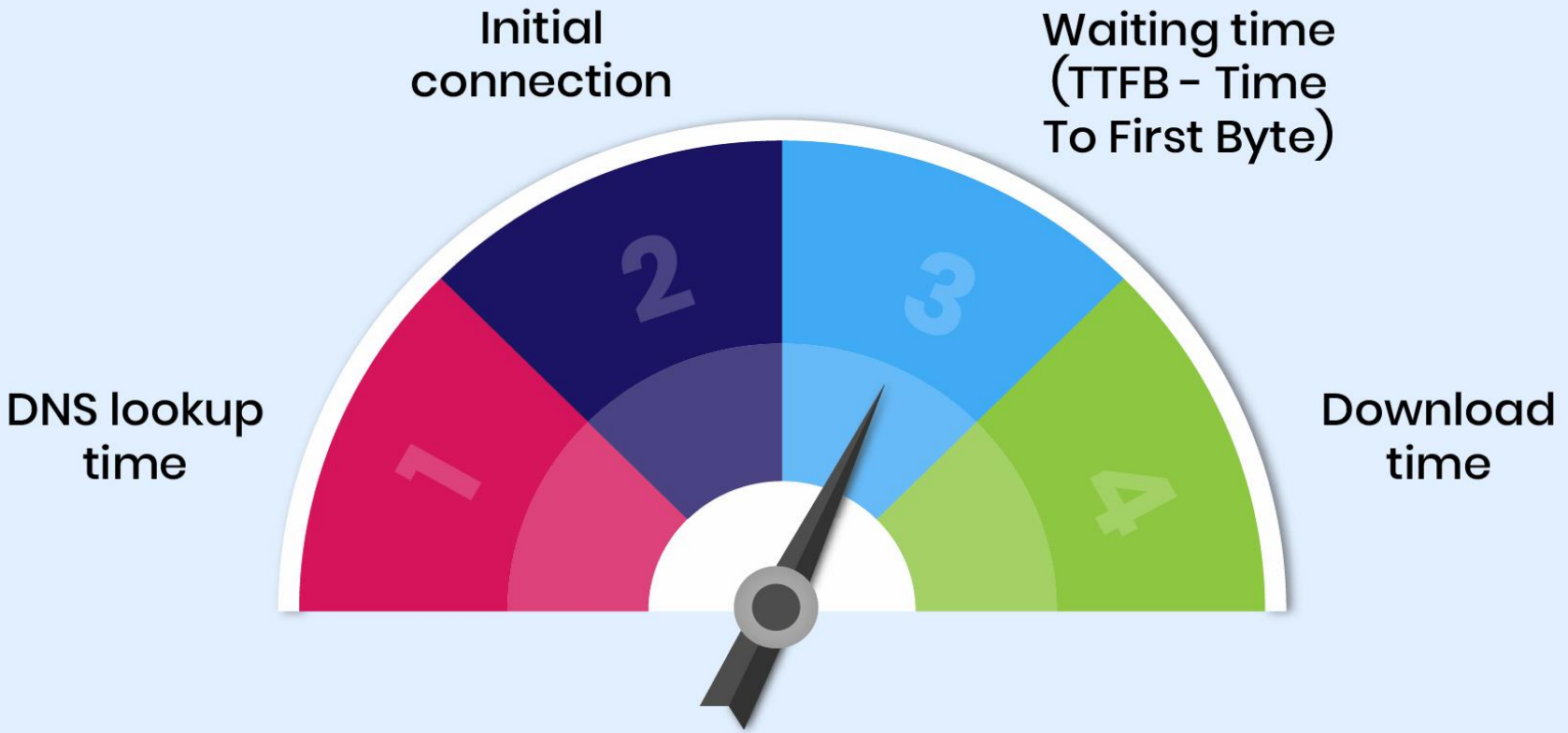
Adding Form Validation

JavaScript provides a way to validate form's data on the client's computer before sending it to the web server.



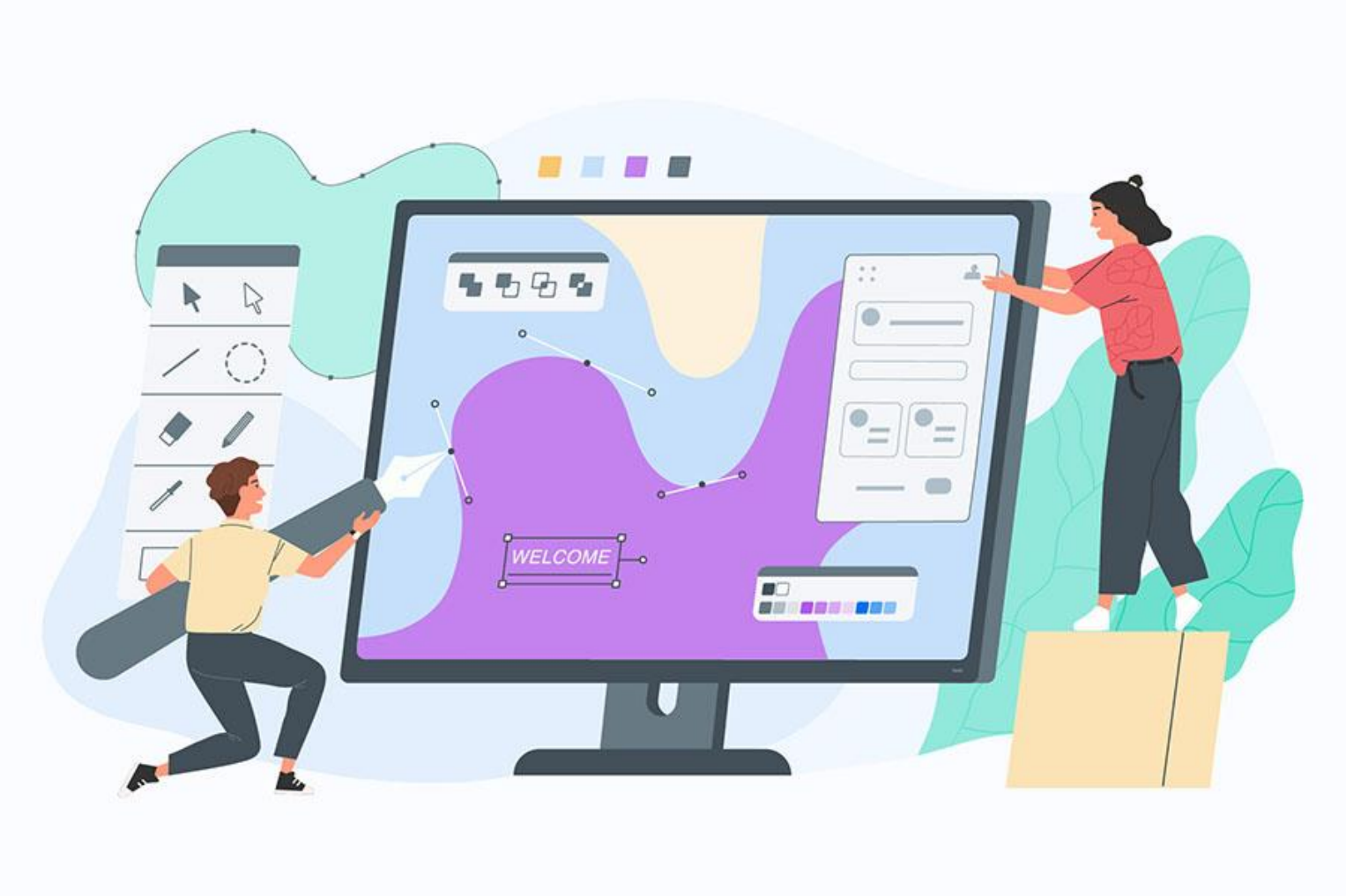
Websites Performance

Reviewing the performance of websites. Website performance refers to **how quickly a website loads on a web browser**. It also defines the quality of the website's usability, interactivity, and reliability. Web performance is often a visitor's first assessment of our site. If a site loads quickly, the visitor can start interacting with it sooner, improving the **user experience**.



Adding Visual Design Elements

Adding visual design elements like colors and fonts. There are 7 visual elements in total, they are line, shape, color, value, form, texture, and space.



Website Features More Functional

Making website features more functional.

25 features for e-commerce website.

Resolving Any Issues

Menyelesaikan masalah apa pun yang ditemui *end-users* di website.

This page isn't working

www.technewstoday.com redirected you too many times.

Try clearing your cookies

ERR_TOO_MANY_REDIRECTS

Reload

2 – JavaScript Fundamentals

How *to* execute Javascript?

JavaScript is a text-based language that does not need any conversion before being executed. Other languages like Java and C++ need to be compiled to be executable but JavaScript is executed instantly by a type of program that interprets the code called a parser (*pretty much all web browsers contain a JavaScript parser*).

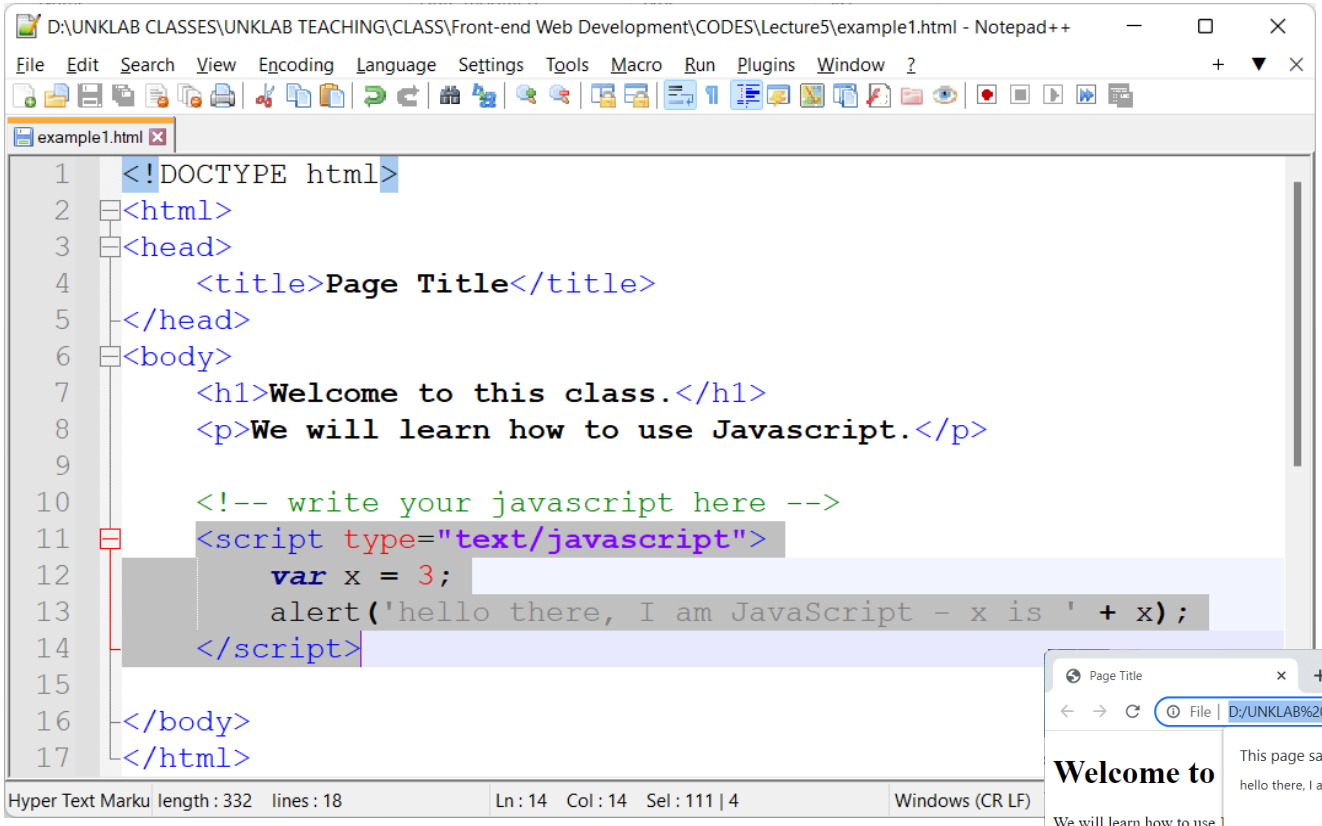
To execute JavaScript in a browser you have two options:

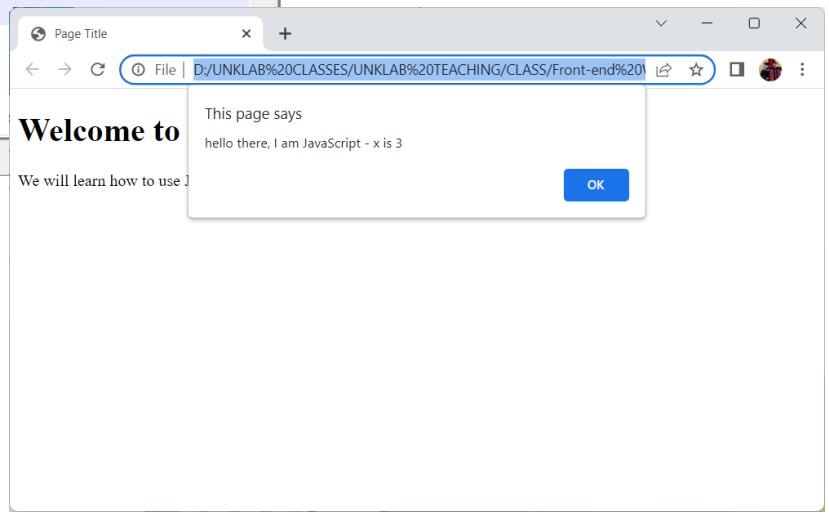
(1) put *it* inside a <script> element anywhere inside an HTML document, or

(2) put *it* inside an external JavaScript file (*with a .js extension*) and then reference that file inside the HTML document using an empty <script> element with a *src* attribute.

```
<script type="text/javascript" src="config.js"></script>
<script type="text/javascript" src="base.js"></script>
<script type="text/javascript" src="effects.js"></script>
<script type="text/javascript" src="validation.js"></script>
<script type="text/javascript" src="widgets.js"></script>
```

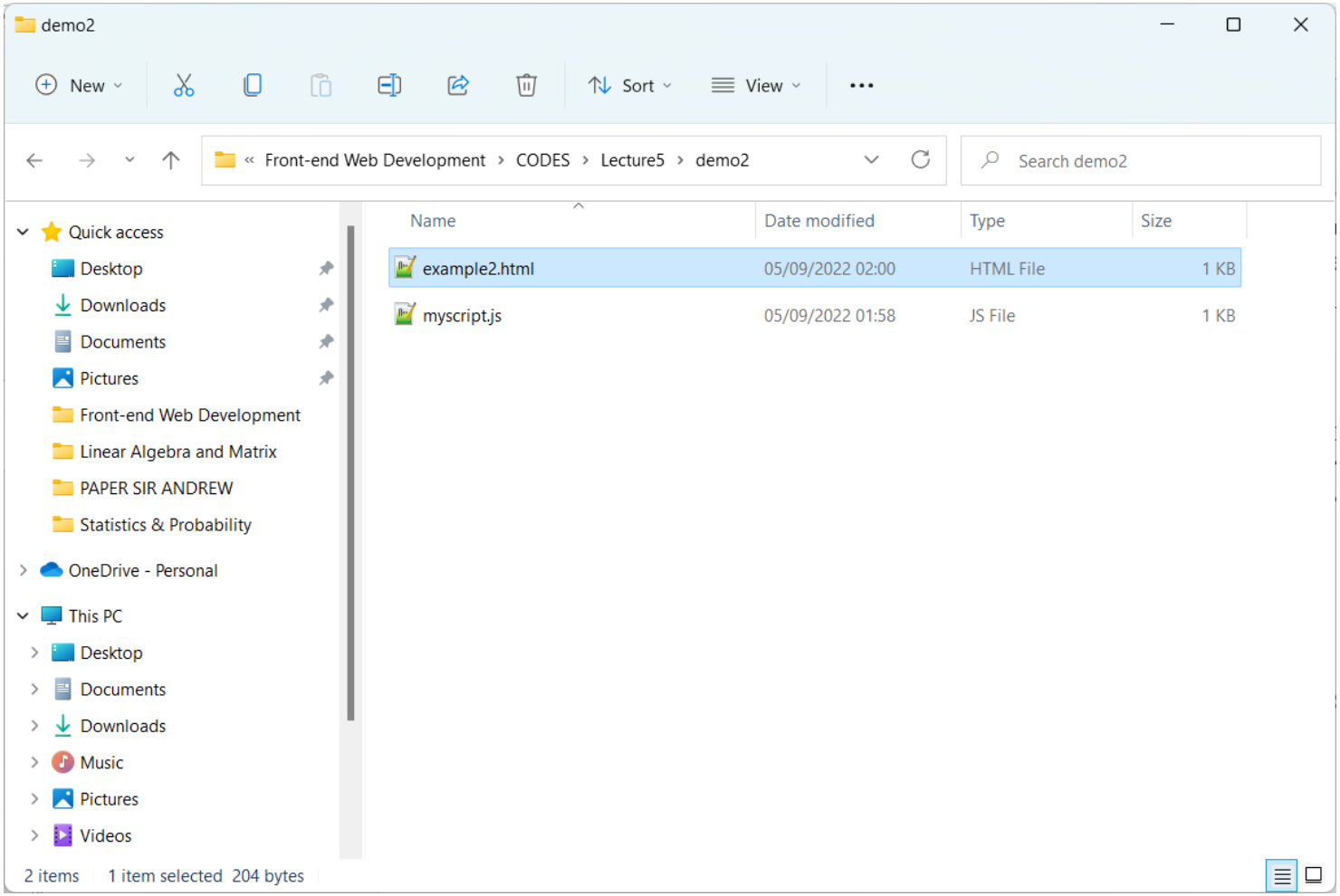
(1) Include JavaScript Inside HTML





(2) External JavaScript File

Create two files in your directory.



(2) External JavaScript File

Write down these Javascript and HTML codes. The classic best practice for placing scripts was in the *head* of the document:

D:\UNKLAB CLASSES\UNKLAB TEACHING\CLASS\Front-end Web Development\CODES\Lecture5\demo2\myscript.js - Notepad...

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

myscript.js example2.html

```
1  // test display Name
2  var name = "Semmy Taju";
3  alert('Hi, my name is '+name);
```

JavaScript file length : 78 lines : 3 Ln : 3 Col : 29 Pos : 77

D:\UNKLAB CLASSES\UNKLAB TEACHING\CLASS\Front-end Web Development\CODES\Lecture5\demo2\example2.html - Note...

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

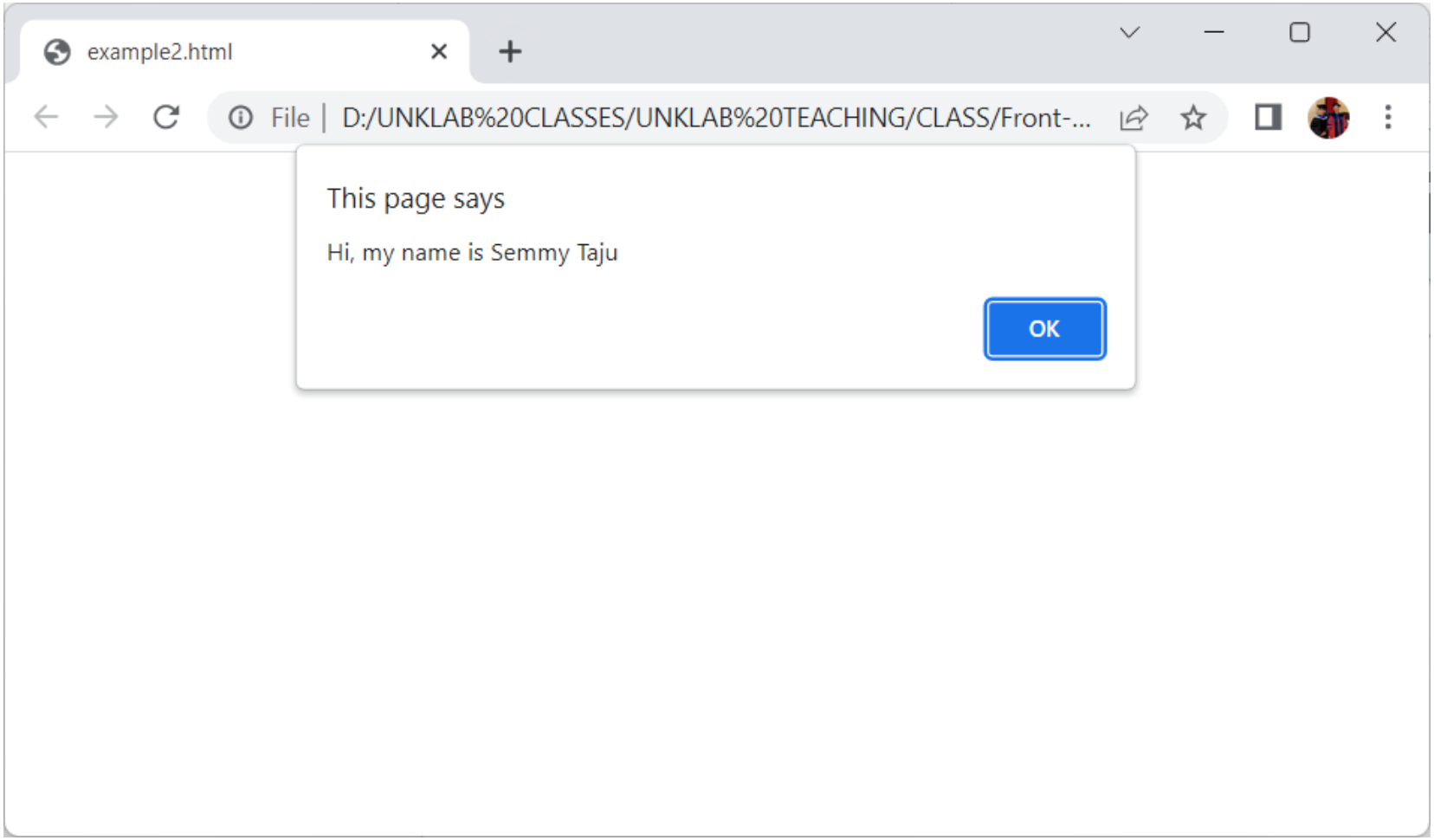
myscript.js example2.html

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <title></title>
6      <script src="myscript.js"></script>
7    </head>
8    <body>
9      <!-- this is a comment in HTML -->
10   </body>
11 </html>
```

Hyper Text Marku length : 204 lines : 11 Ln : 6 Col : 26 Pos : 110 Windows (CR LF) UTF-8 INS

(2) External JavaScript File

Output program should display your name using alert message dialog.



3 – JavaScript Statements

JavaScript Keywords

Keyword	Description
var	Declares a variable
let	Declares a block variable
const	Declares a block constant
if	Marks a block of statements to be executed on a condition
switch	Marks a block of statements to be executed in different cases
for	Marks a block of statements to be executed in a loop
function	Declares a function
return	Exits a function
try	Implements error handling to a block of statements

- JavaScript statements often start with a keyword to identify the JavaScript action to be performed.

- Let dan Const menganut sistem *block scope*, yang mana cakupan variabelnya hanya bisa diakses di dalam blocknya saja. Var menganut sistem functional scope, yang mana variabelnya dapat diakses dari dalam maupun dari luar block kecuali di luar function.

Semicolons in JavaScript Statements

Semicolons separate JavaScript statements. Add a semicolon at the end of each executable statement. When separated by semicolons, multiple statements on one line are allowed.

```
<!DOCTYPE html>
<html>
<body>
  <h2>JavaScript Statements</h2>
  <p>JavaScript statements are separated by semicolons.</p>
  <p id="demo1"></p>

  <script>
    let a, b, c;
    a = 5;
    b = 6;
    c = a + b;
    document.getElementById("demo1").innerHTML = c;
  </script>

</body>
</html>
```

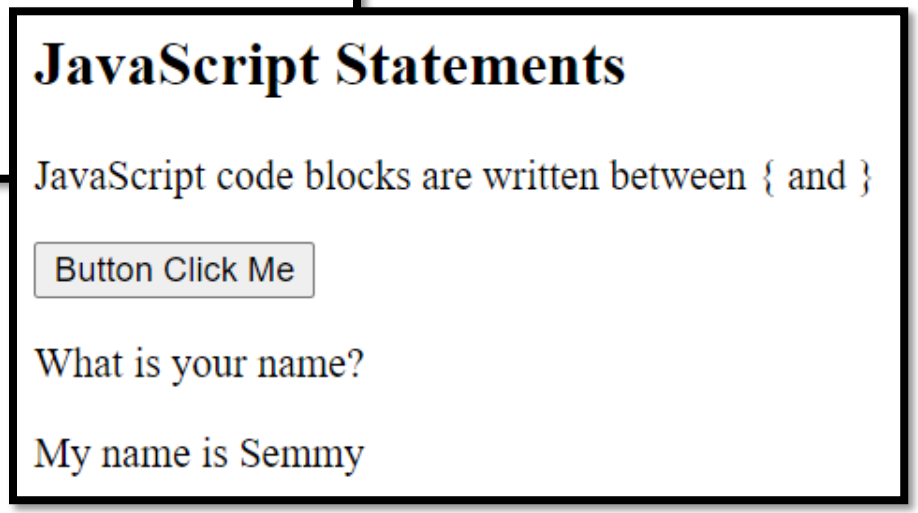
JavaScript Code Blocks

JavaScript statements can be grouped together in code blocks, inside curly brackets {...}. The purpose of code blocks is to define statements to be executed together.

```
<!DOCTYPE html>
<html>
<body>
  <h2>JavaScript Statements</h2>
  <p>JavaScript code blocks are written between { and }</p>
  <button type="button" onclick="myFunction()">Button Click Me</button>
  <p id="demo1"></p>
  <p id="demo2"></p>

  <script>
    function myFunction() {
      document.getElementById("demo1").innerHTML = "What is your name?";
      document.getElementById("demo2").innerHTML = "My name is Semmy";
    }
  </script>

</body>
</html>
```



JavaScript Comments

- Not all JavaScript statements are "executed".
- Code after double slashes // or between /* and */ is treated as a comment.

```
<!DOCTYPE html>
<html>
<body>
  <h2>JavaScript Comments are NOT Executed</h2>
  <p id="demo"></p>

  <script>
    let x;
    x = 5;
    // x = 6; not executed
    document.getElementById("demo").innerHTML = x;
  </script>

</body>
</html>
```

4 – JavaScript Variables

JavaScript Variables

Variables are containers for storing data (*storing data values*). In this example, x, y, and z, are variables, declared with the *var* keyword.

4 Ways to Declare a JavaScript Variable:

1) Using *var*
2) Using *let*
3) Using *const*
4) Using nothing

Example

```
var x = 5;
var y = 6;
var z = x + y;
```

Example

```
let x = 5;
let y = 6;
let z = x + y;
```

Example

```
x = 5;
y = 6;
z = x + y;
```

Example

```
const price1 = 5;
const price2 = 6;
let total = price1 + price2;
```

JavaScript Types *are* Dynamic

JavaScript has dynamic types. This means that the same variable can be used to hold different data types.

```
let x;               // Now x is undefined
x = 5;               // Now x is a Number
x = "John";          // Now x is a String
```

```
let carName1 = "Volvo XC60";   // Using double quotes
let carName2 = 'Volvo XC60';   // Using single quotes
```

```
let x = 5;
let y = 5;
let z = 6;
(x == y)       // Returns true
(x == z)       // Returns false
```

JavaScript Arrays

Why Use Arrays?

If you have a list of items (a list of car names, for example), storing the cars in single variables could look like this:

```
let car1 = "Saab";
let car2 = "Volvo";
let car3 = "BMW";
```

However, what if you want to loop through the cars and find a specific one? And what if you had not 3 cars, but 300?

The solution is an array!

An array can hold many values under a single name, and you can access the values by referring to an index number.

Creating an Array

Using an array literal is the easiest way to create a JavaScript

Syntax:

```
const array_name = [item1, item2, ...];
```

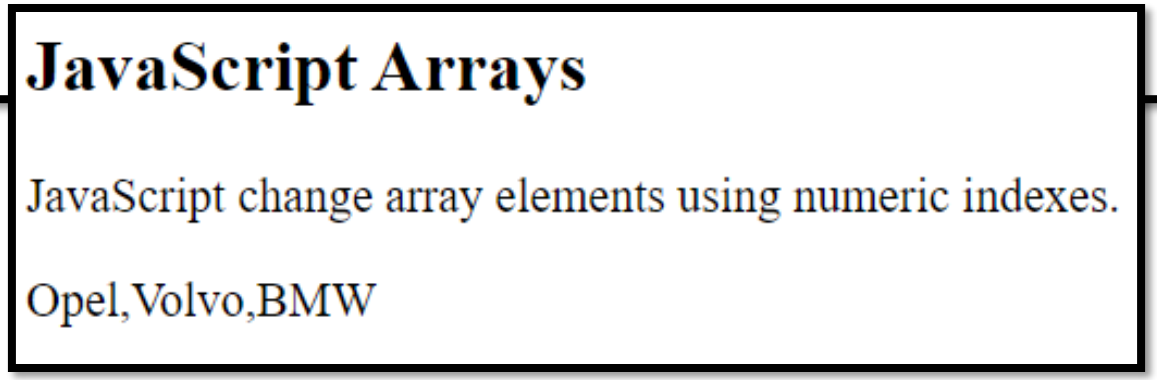
```
<!DOCTYPE html>
<html>
<body>
  <h2>JavaScript Arrays</h2>
  <p id="demo"></p>

  <script>
    const cars = [];
    cars[0]= "Saab";
    cars[1]= "Volvo";
    cars[2]= "BMW";
    document.getElementById("demo").innerHTML = cars;
  </script>
</body>
</html>
```

Changing *an* Array Element

```
<!DOCTYPE html>
<html>
<body>
  <h2>JavaScript Arrays</h2>
  <p>JavaScript change array elements using numeric indexes.</p>
  <p id="demo"></p>

  <script>
    const cars = ["Saab", "Volvo", "BMW"];
    cars[0] = "Opel";
    document.getElementById("demo").innerHTML = cars;
  </script>
</body>
</html>
```



JS Conditional Statements

JavaScript if, else, and else if

Conditional Statements

Very often when you write code, you want to perform different actions for different decisions.

You can use conditional statements in your code to do this.

In JavaScript we have the following conditional statements:

- Use `if` to specify a block of code to be executed, if a specified condition is true
- Use `else` to specify a block of code to be executed, if the same condition is false
- Use `else if` to specify a new condition to test, if the first condition is false
- Use `switch` to specify many alternative blocks of code to be executed

The else if Statement

```
<!DOCTYPE html>
<html>
<body>
  <h2>JavaScript if-else Statements</h2>
  <p>Greeting before start class.</p>
  <p id="demo"></p>

  <script>
    const time = new Date().getHours();
    let greeting;
    if (time < 10) {
      greeting = "Good morning";
    } else if (time < 20) {
      greeting = "Good day";
    } else {
      greeting = "Good evening";
    }
    document.getElementById("demo").innerHTML = greeting;
  </script>

</body>
</html>
```

JavaScript For Loop

Different Kinds of Loops

JavaScript supports different kinds of loops:

- `for` - loops through a block of code a number of times
- `for/in` - loops through the properties of an object
- `for/of` - loops through the values of an iterable object
- `while` - loops through a block of code while a specified condition is true
- `do/while` - also loops through a block of code while a specified condition is true

The For Loop

The `for` statement creates a loop with 3 optional expressions:

```
for (expression 1; expression 2; expression 3) {
  // code block to be executed
}
```

JavaScript For Loop

```
<!DOCTYPE html>
<html>
<body>
  <h2>JavaScript For Loop</h2>
  <p id="demo"></p>

  <script>
  const cars = ["BMW", "Volvo", "Saab", "Ford"];
  let i = 0;
  let len = cars.length;
  let text = "";

  for (; i < len; ) {
    text += cars[i] + "<br>";
    i++;
  }
  document.getElementById("demo").innerHTML = text;
  </script>

</body>
</html>
```

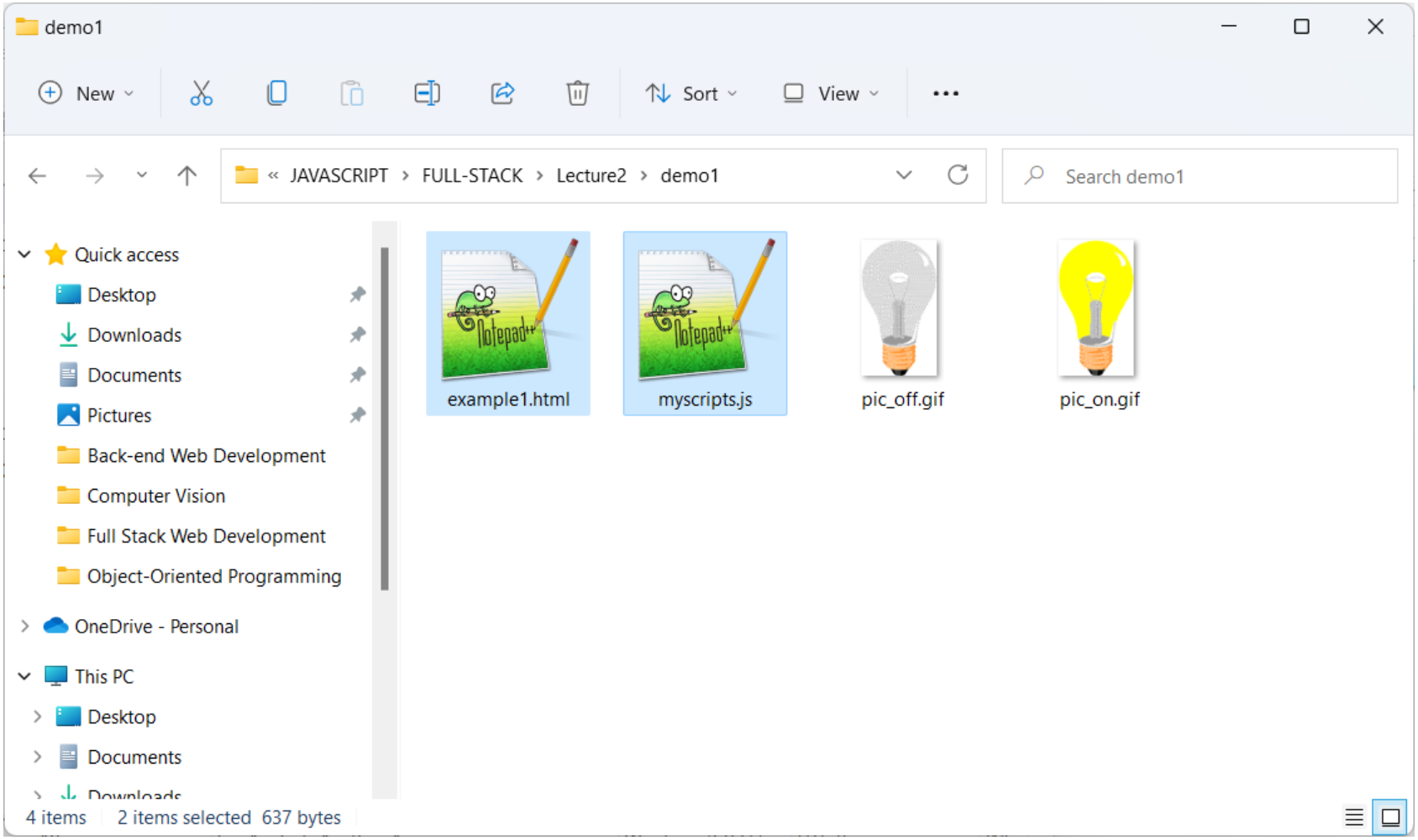
Exercise *for* Students

Exercise #1

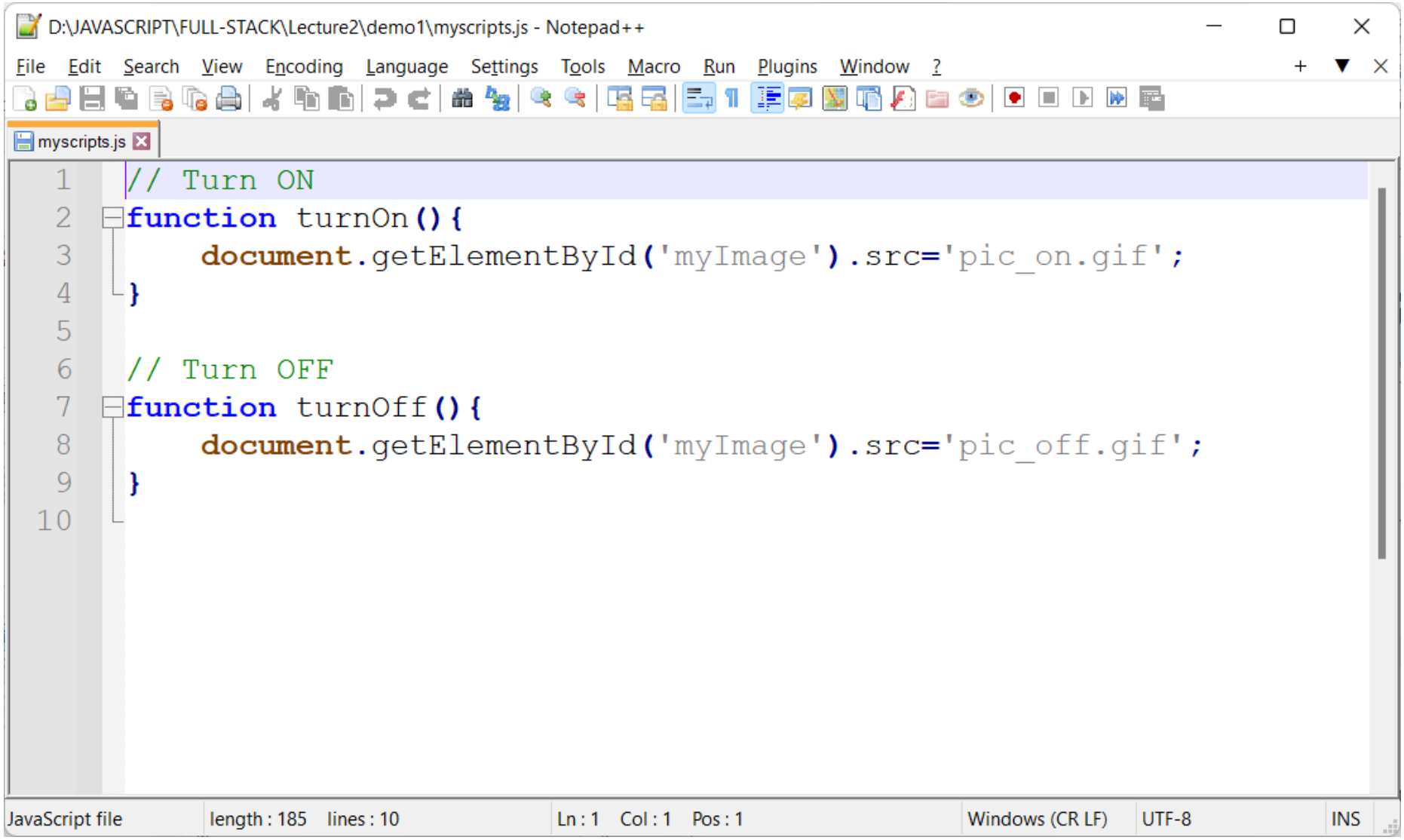
JavaScript changes HTML attribute values *to* turn ON/OFF the light.

Create New Folder

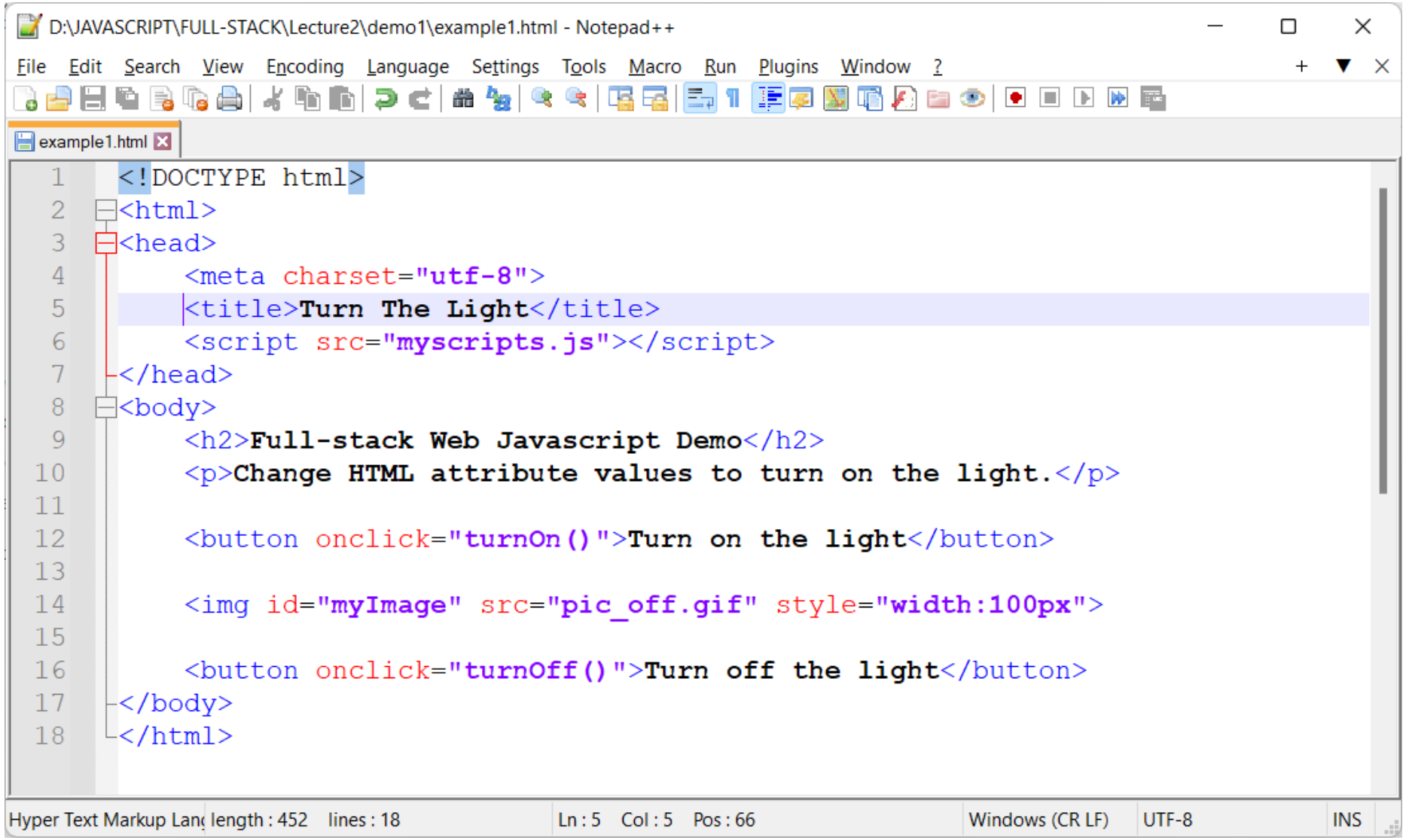
Buat folder baru dengan nama “demo1”.



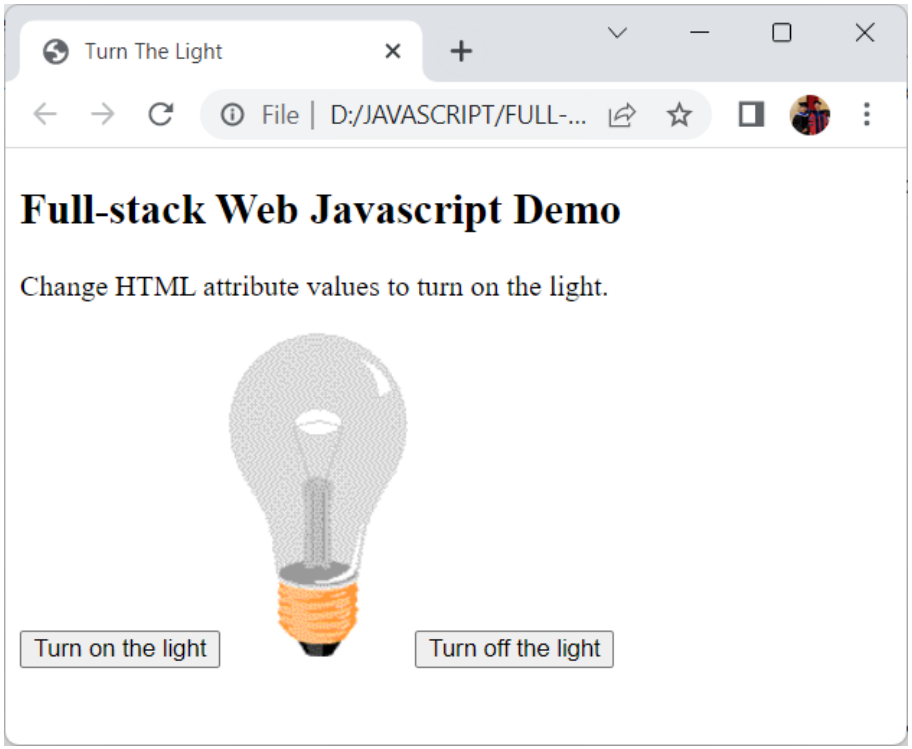
Write Javascript Code

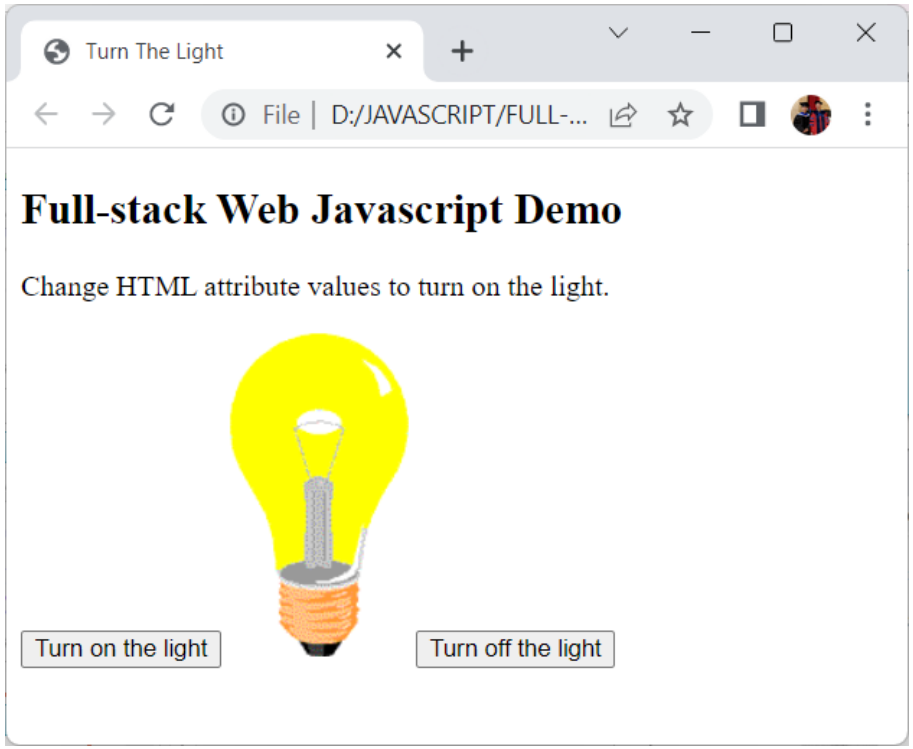


Write HTML Tags



Expected Output



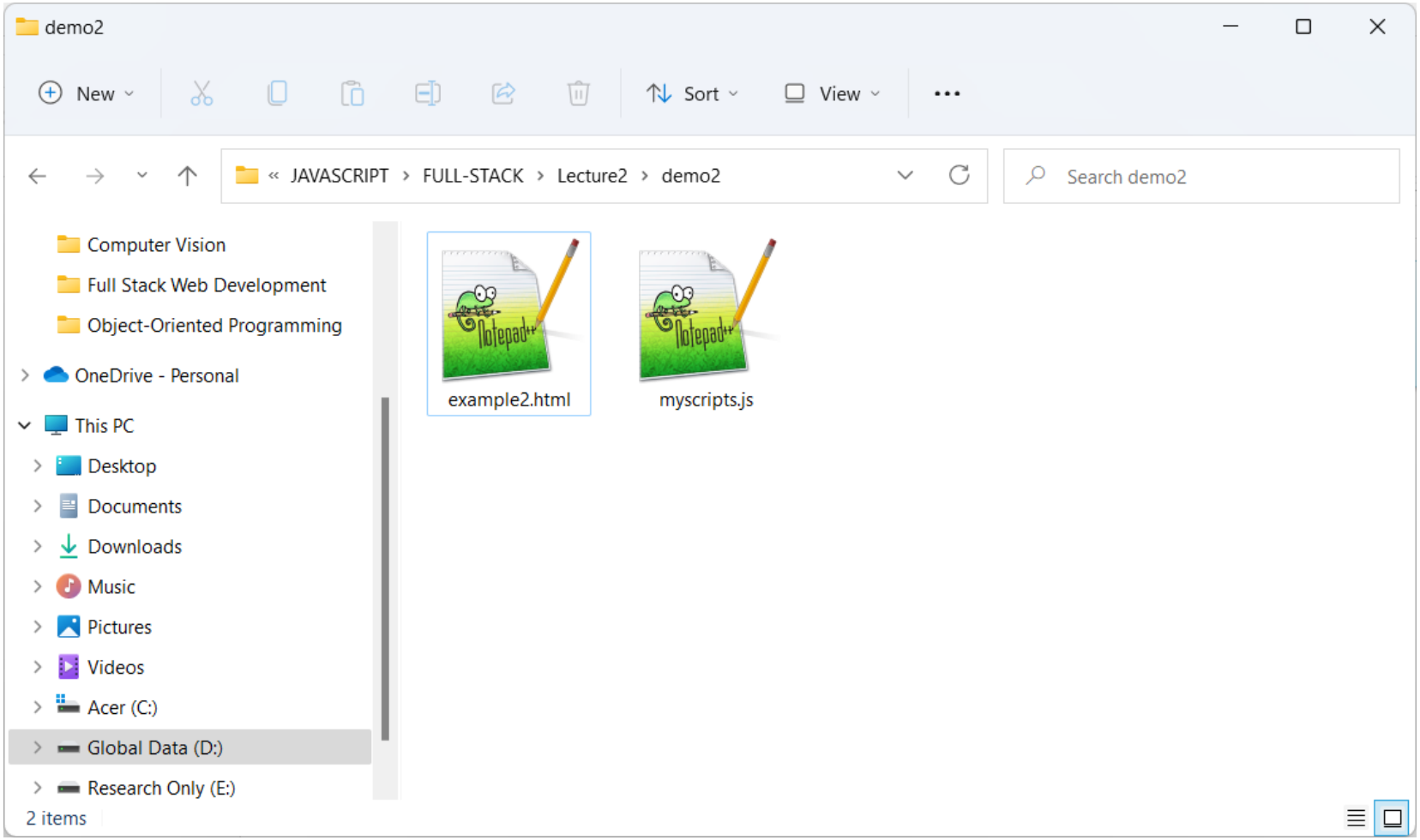


Exercise #2

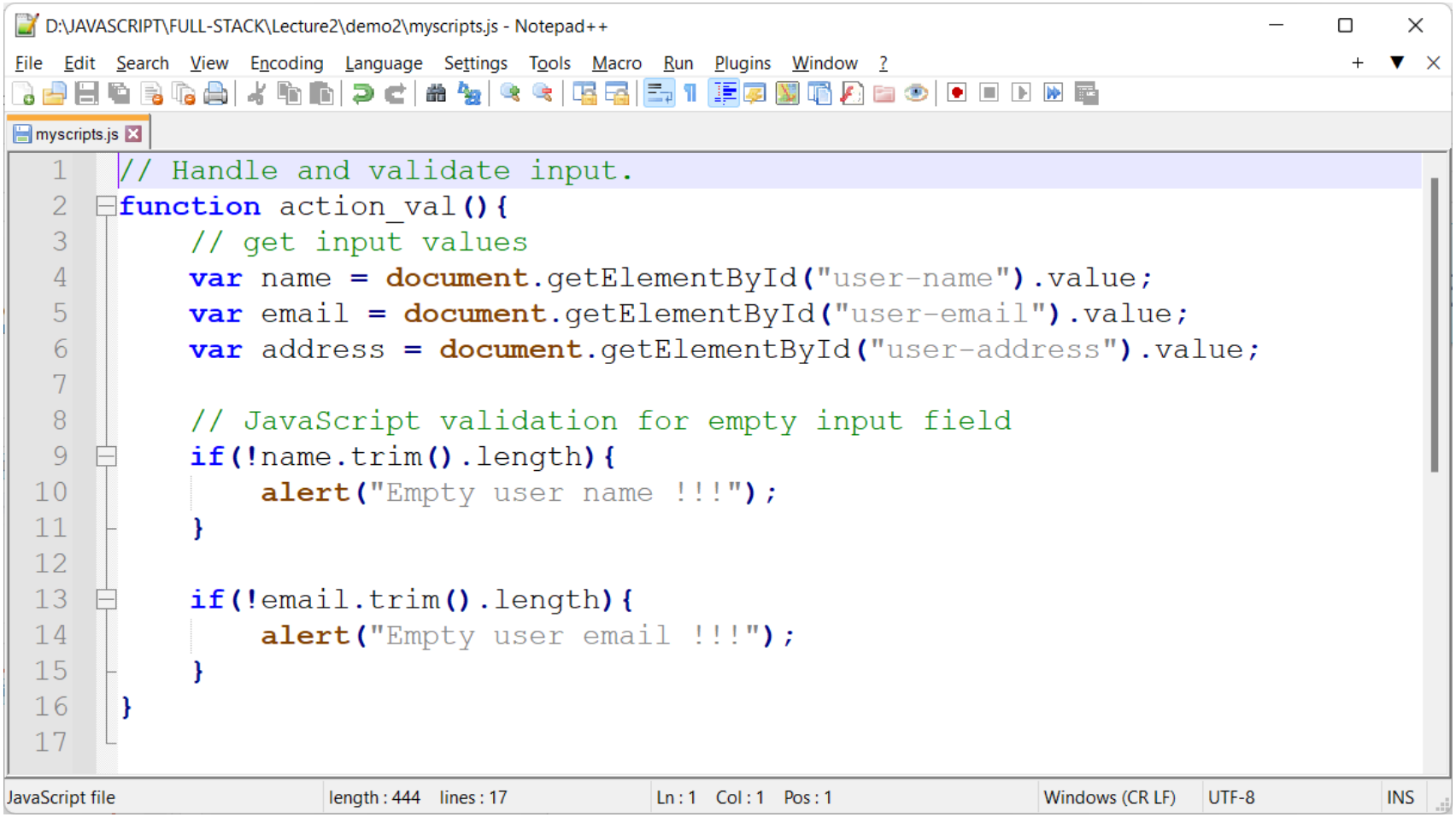
JavaScript Form Validation Example.

Create New Folder

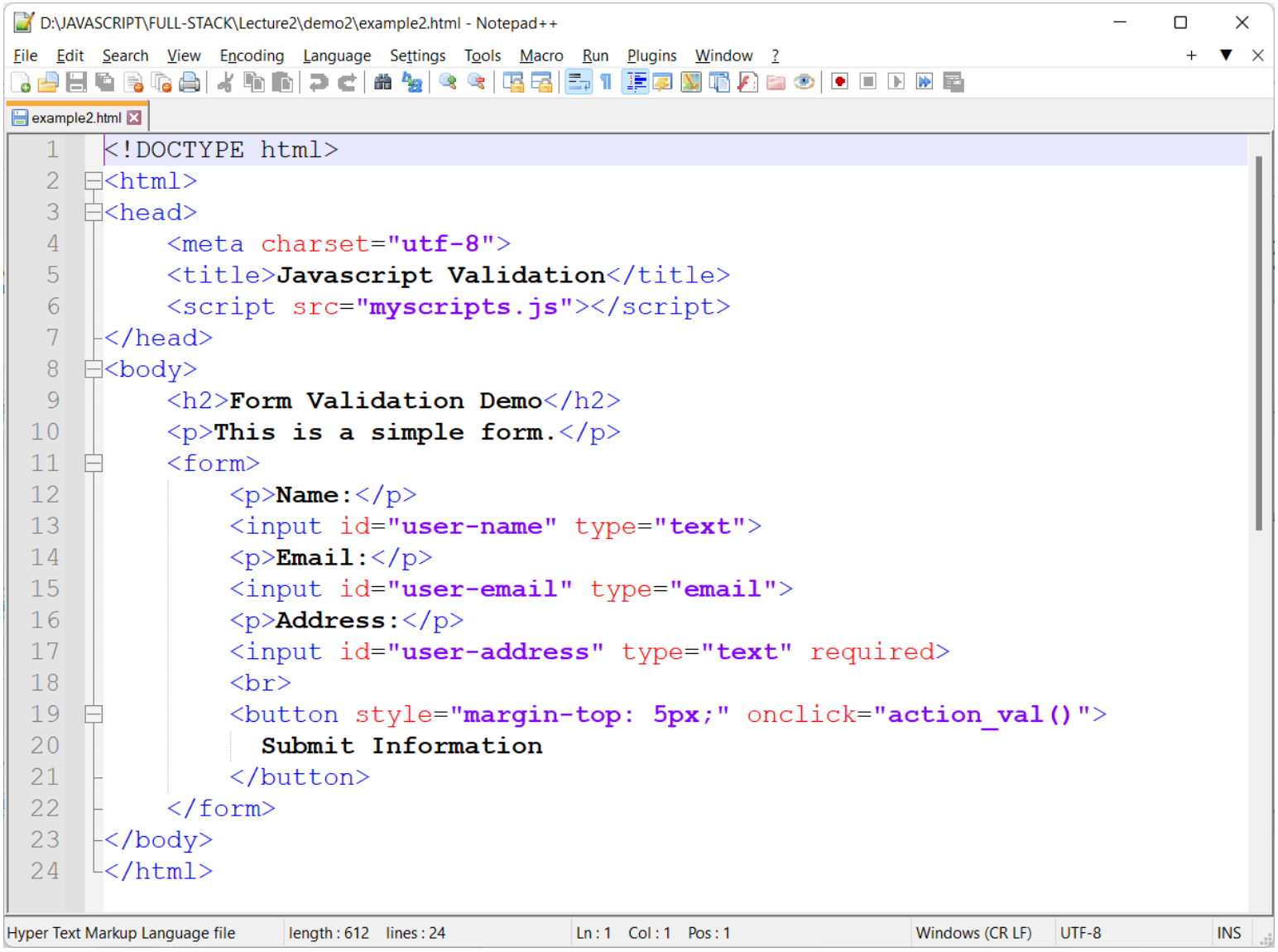
Buat folder baru dengan nama “demo2”.



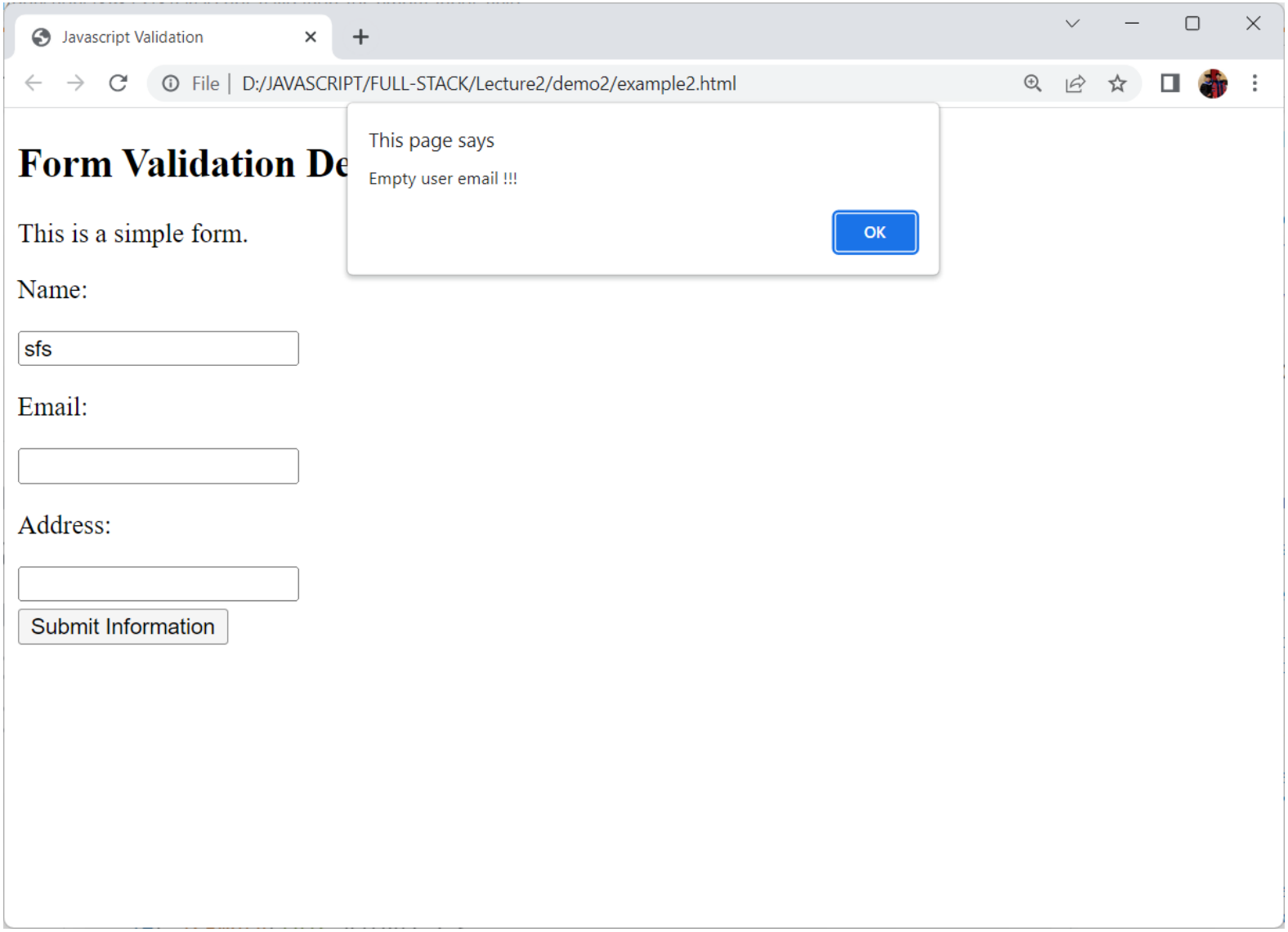
Write JavaScript Code



Write HTML Tags



Expected Output

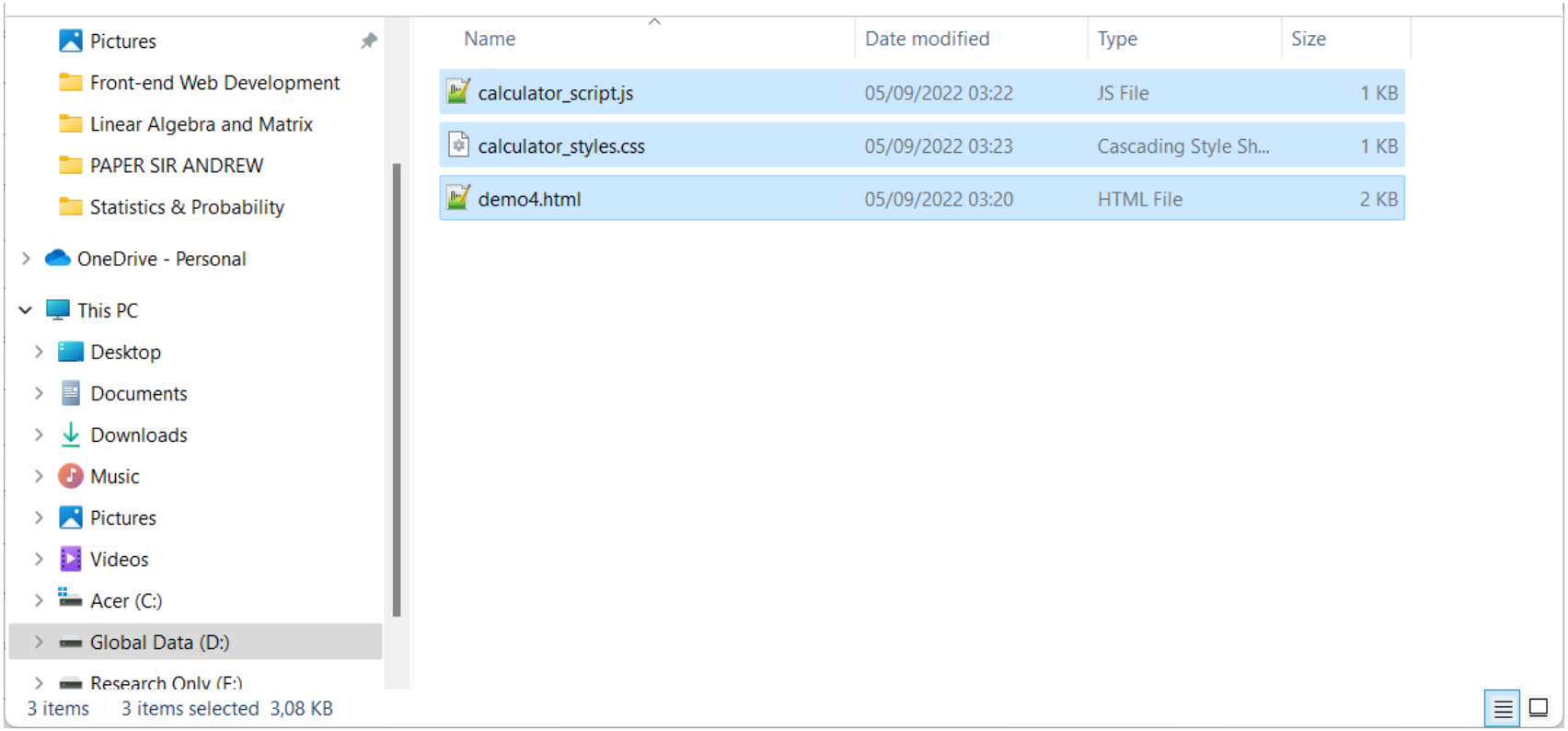


Exercise #3

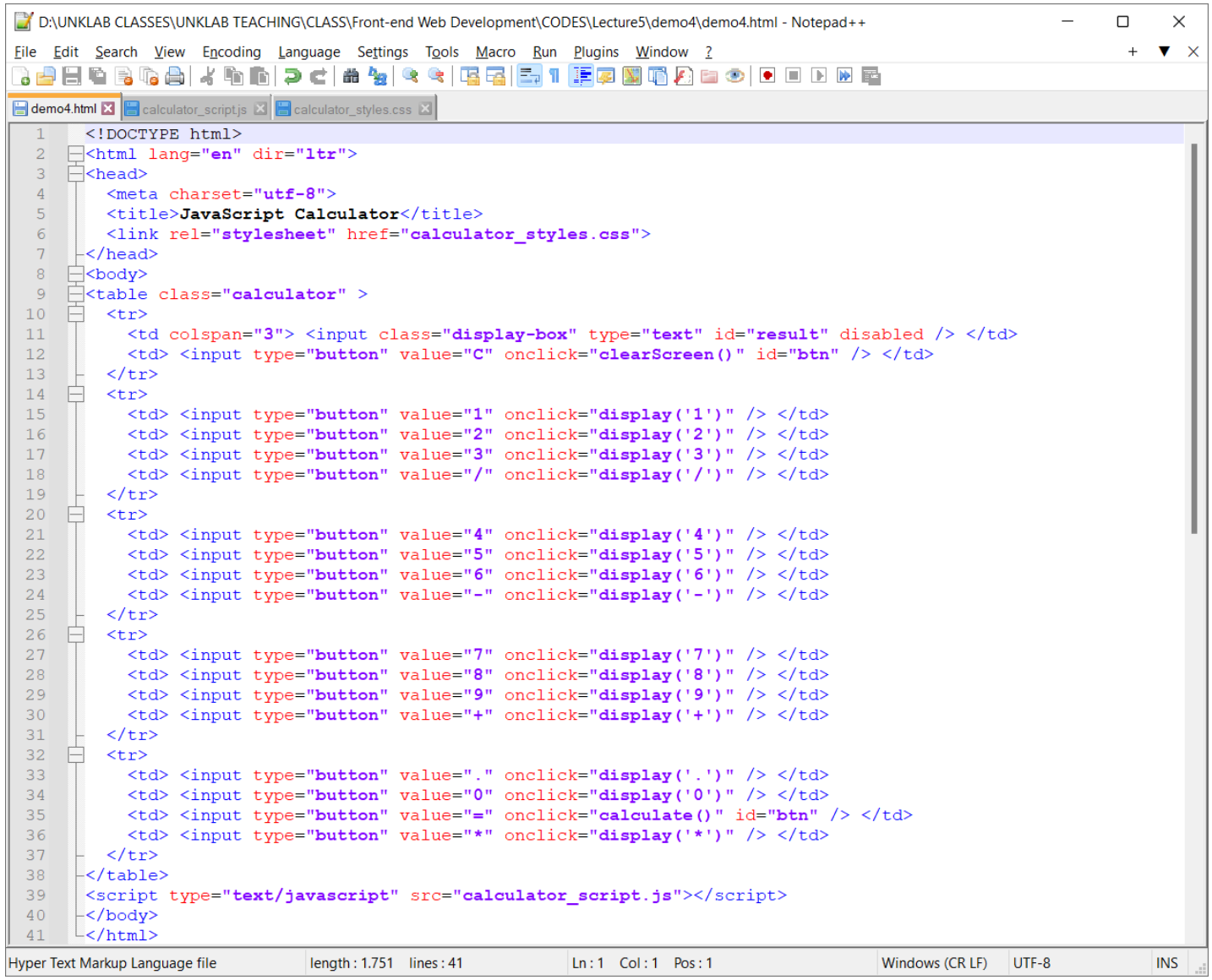
HTML, JavaScript and CSS to create simple calculator.

Create New Folder & Files

Buat folder baru untuk menyimpan 3 file dengan masing-masing nama file dibawah.



Write HTML Tags



Write CSS Code *for* Styling

D:\UNKLAB CLASSES\UNKLAB TEACHING\CLASS\Front-end Web Development\CODES\Lecture5\demo4\calculator_styles.css - Notepad++

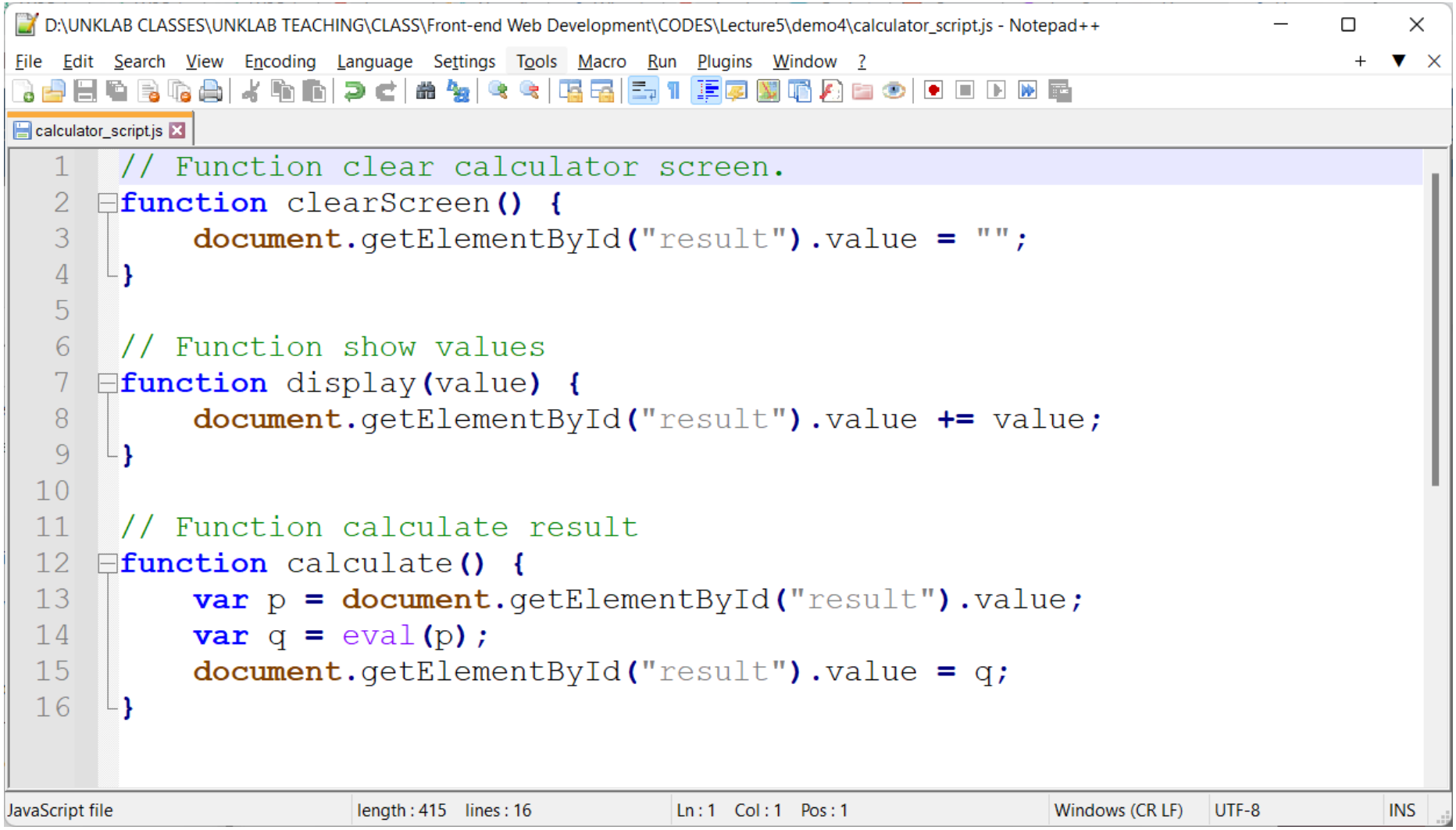
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

calculator_styles.css | calculator_script.js

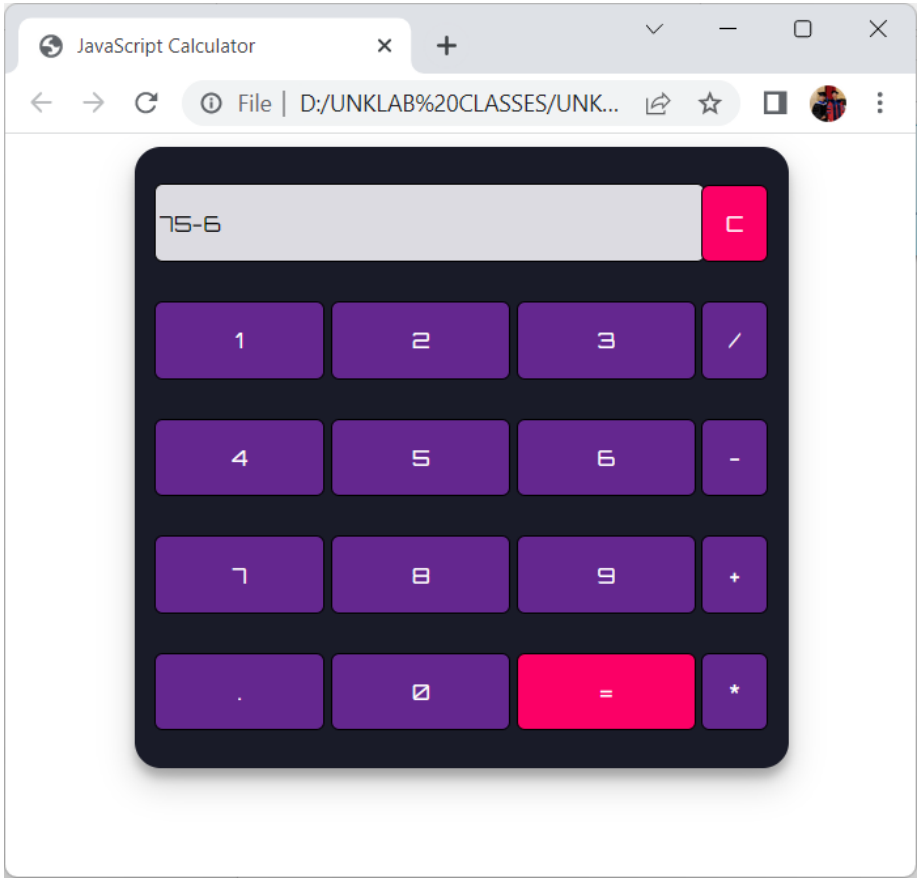
```
@import url('https://fonts.googleapis.com/css2?family=Orbitron&display=swap');
.calculator {
    padding: 10px;
    border-radius: 1em;
    height: 380px;
    width: 400px;
    margin: auto;
    background-color: #191b28;
    box-shadow: rgba(0, 0, 0, 0.19) 0px 10px 20px, rgba(0, 0, 0, 0.23) 0px 6px 6px;
}
.display-box {
    font-family: 'Orbitron', sans-serif;
    background-color: #dcdbe1;
    border: solid black 0.5px;
    color: black;
    border-radius: 5px;
    width: 100%;
    height: 65%;
}
#btn {
    background-color: #fb0066;
}
input[type=button] {
    font-family: 'Orbitron', sans-serif;
    background-color: #64278f;
    color: white;
    border: solid black 0.5px;
    width: 100%;
    border-radius: 5px;
    height: 70%;
    outline: none;
}
input:active[type=button] {
    background: #e5e5e5;
    -webkit-box-shadow: inset 0px 0px 5px #c1c1c1;
    -moz-box-shadow: inset 0px 0px 5px #c1c1c1;
    box-shadow: inset 0px 0px 5px #c1c1c1;
}
```

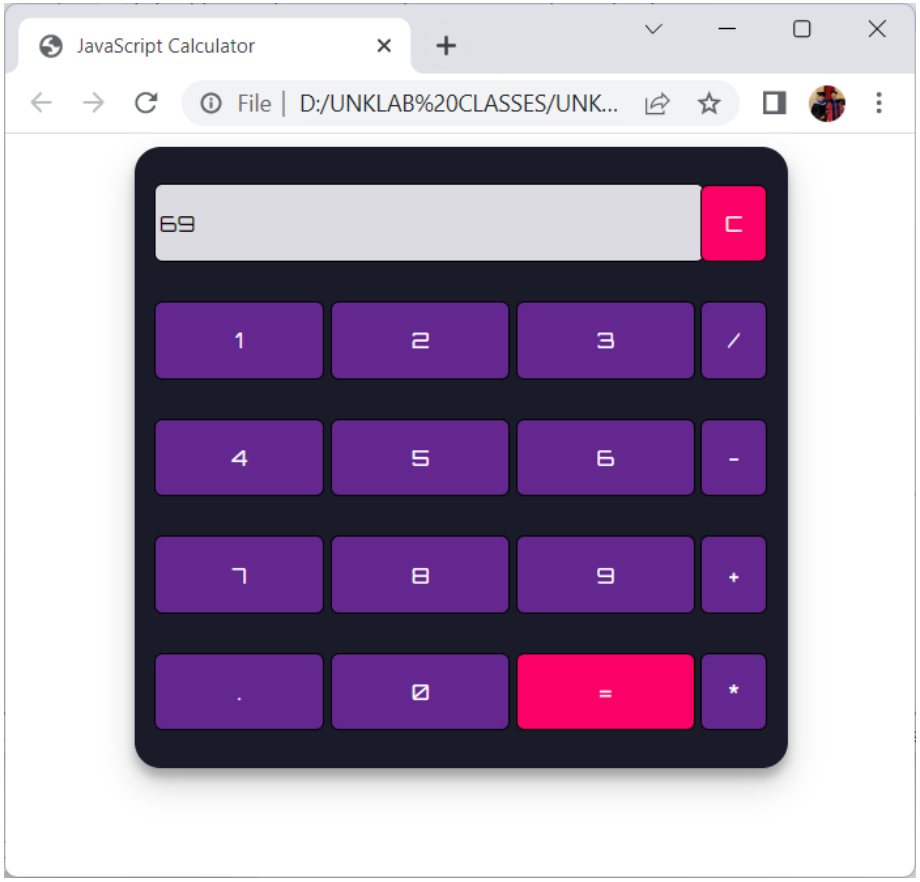
Cascade Style Sheets File | length : 998 lines : 38 | Ln : 1 Col : 1 Pos : 1 | Windows (CR LF) | UTF-8 | INS

Write JavaScript Code



Expected Output



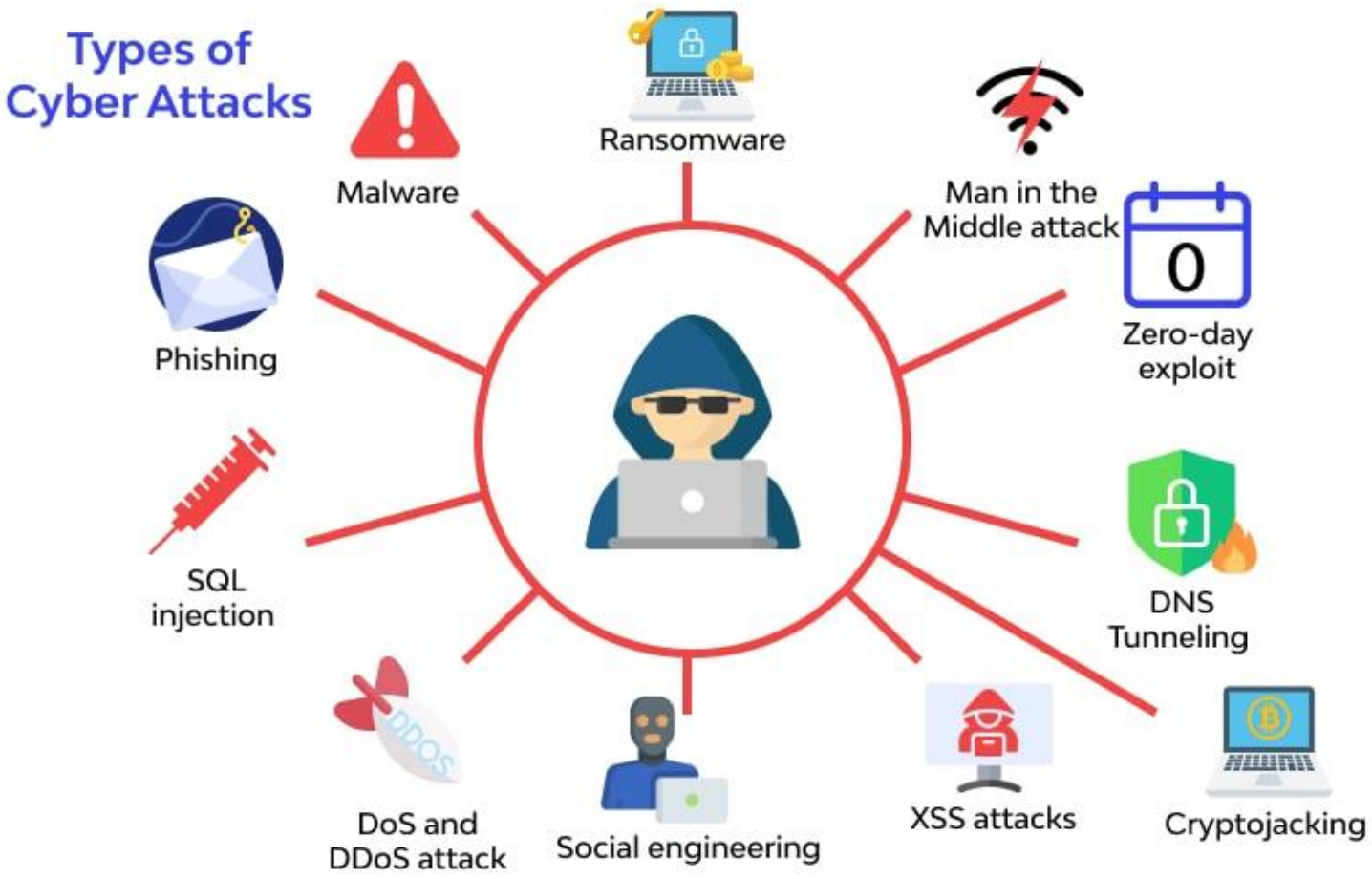


Exercise #4

Cross-Site Scripting (XSS) Attack

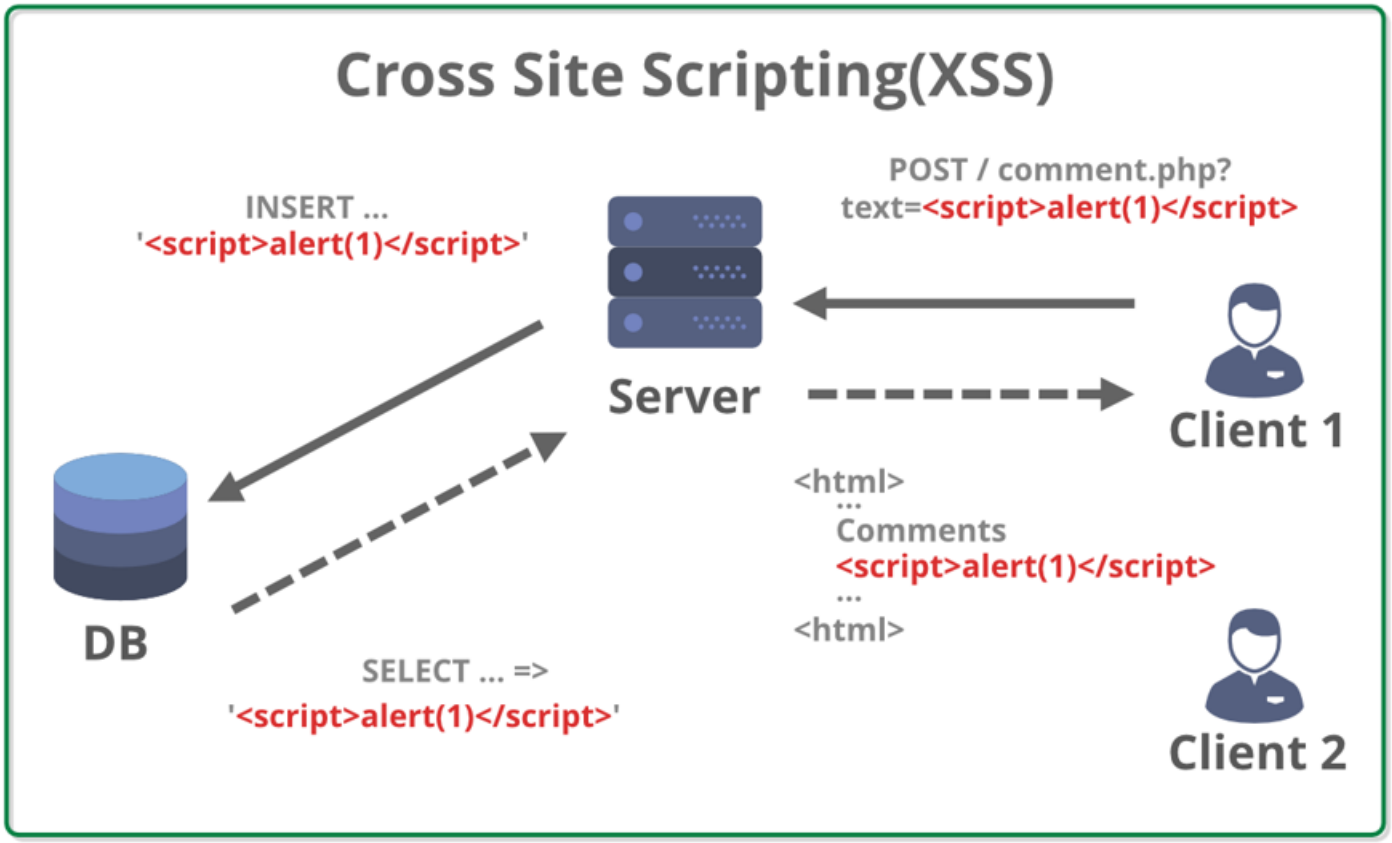
Cyberattacks

Cyberattacks take advantage of weaknesses (*kelemahan*) on the browser side (client-side programming languages) often exploit vulnerabilities (*kerentanan*) in programming languages such as JavaScript, HTML, or CSS.

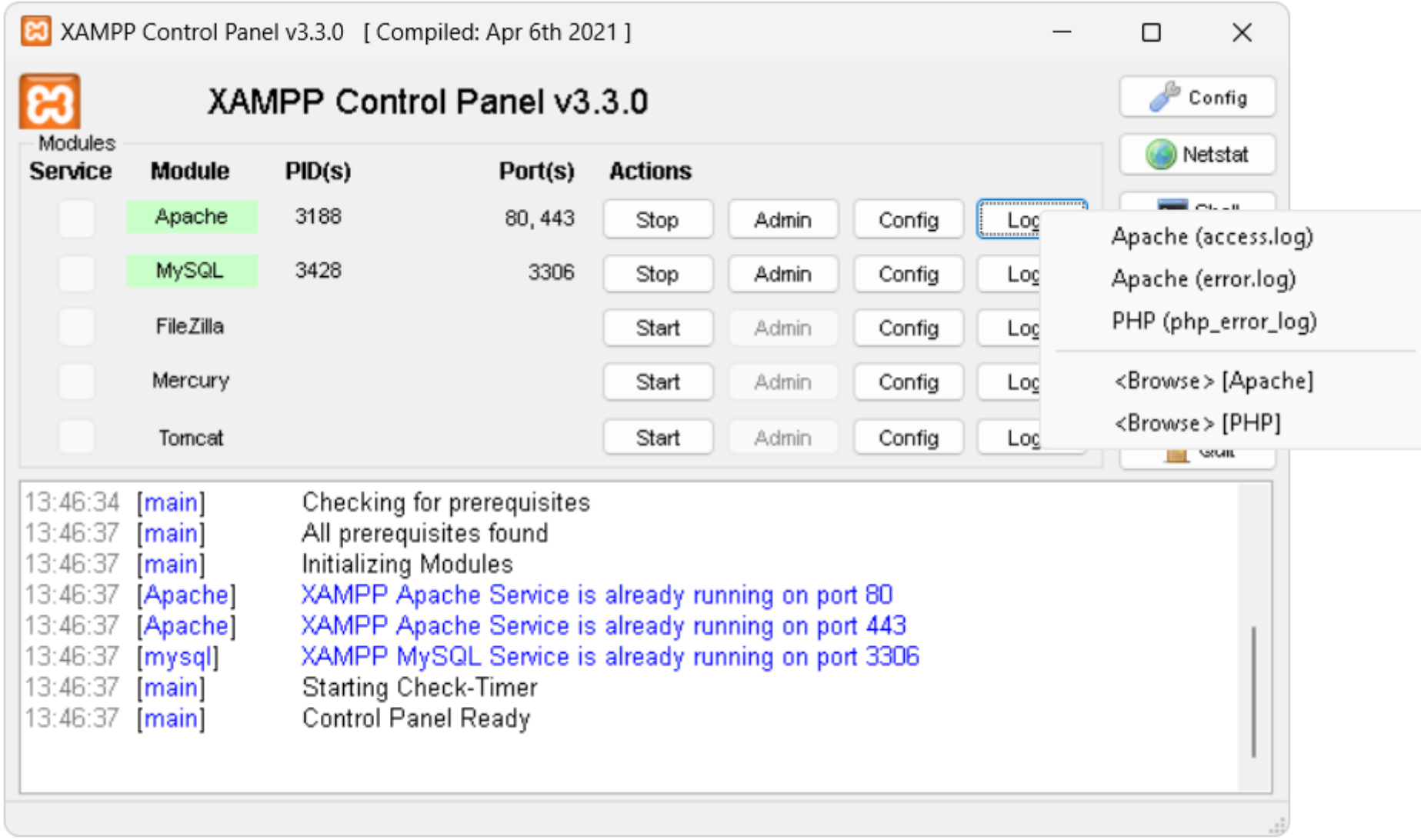


Cross-Site Scripting (XSS) Attack

This attack injects malicious code (usually JavaScript) into a website, which is then executed in the user's browser. The impacts include cookie theft, session hijacking, malware injection, or website appearance manipulation. For example, user input is not properly validated, allowing malicious scripts to be inserted through web forms or URLs.

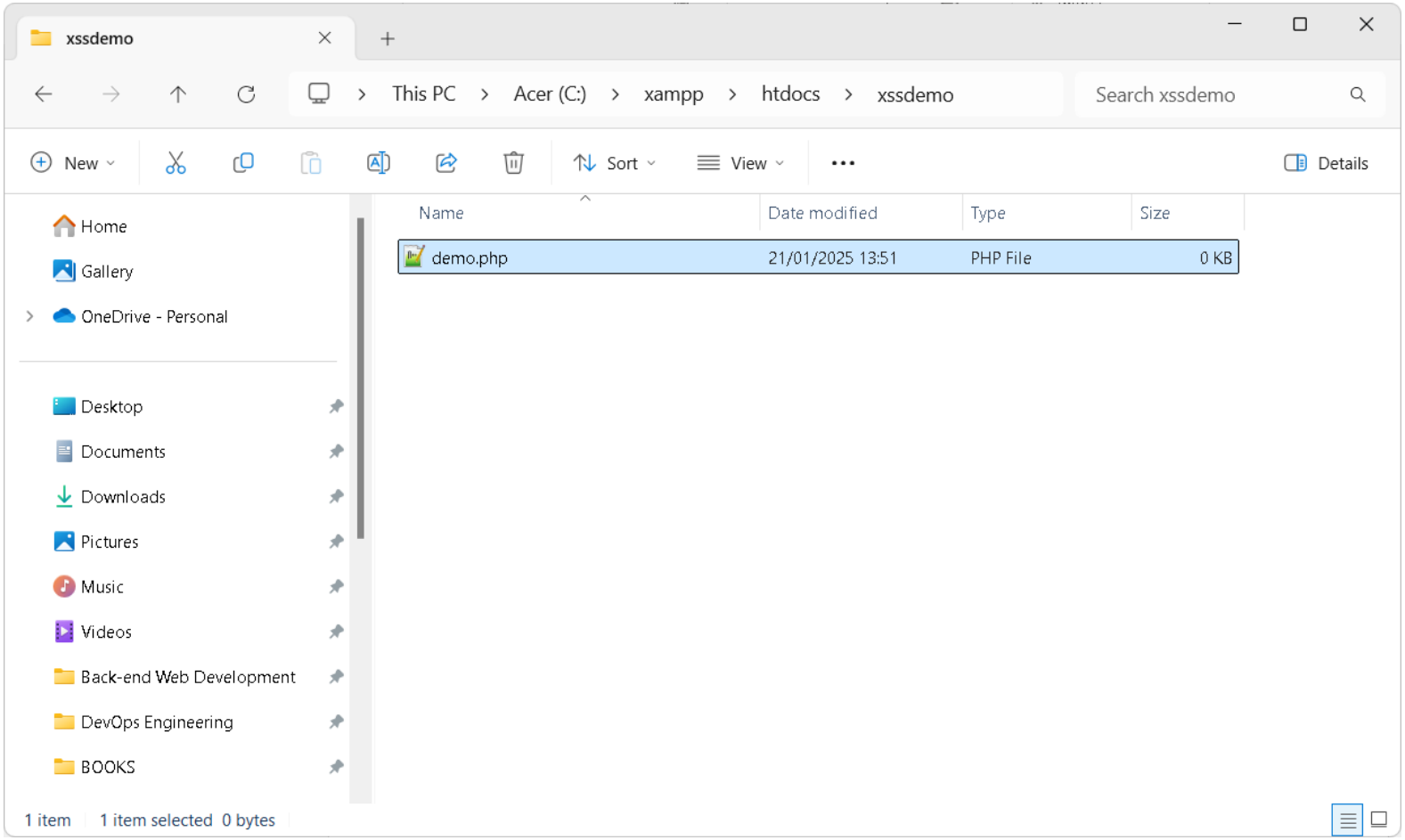


Make Sure XAMPP is Installed



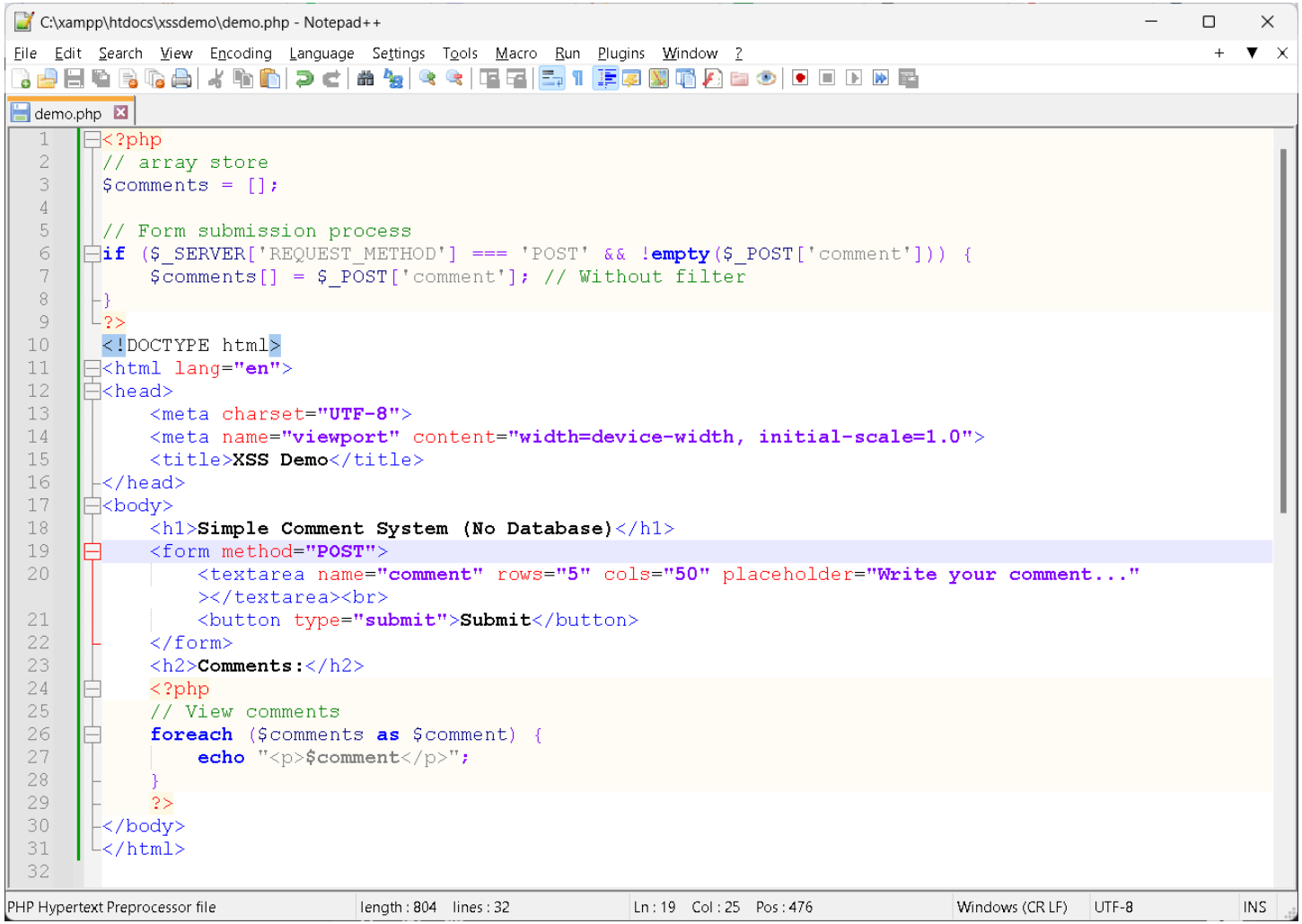
Create Working Directory

Buat folder baru bernama **xssdemo** di dalam direktori **htdocs** (misalnya, **C:/xampp/htdocs/xssdemo**). Selanjutnya, tambahkan file **demo.php** ke dalam folder **xssdemo**.



Modify File demo.php

Perhatikan code pada aplikasi menyimpan data langsung dari *input user* tanpa memfilter atau memvalidasi.

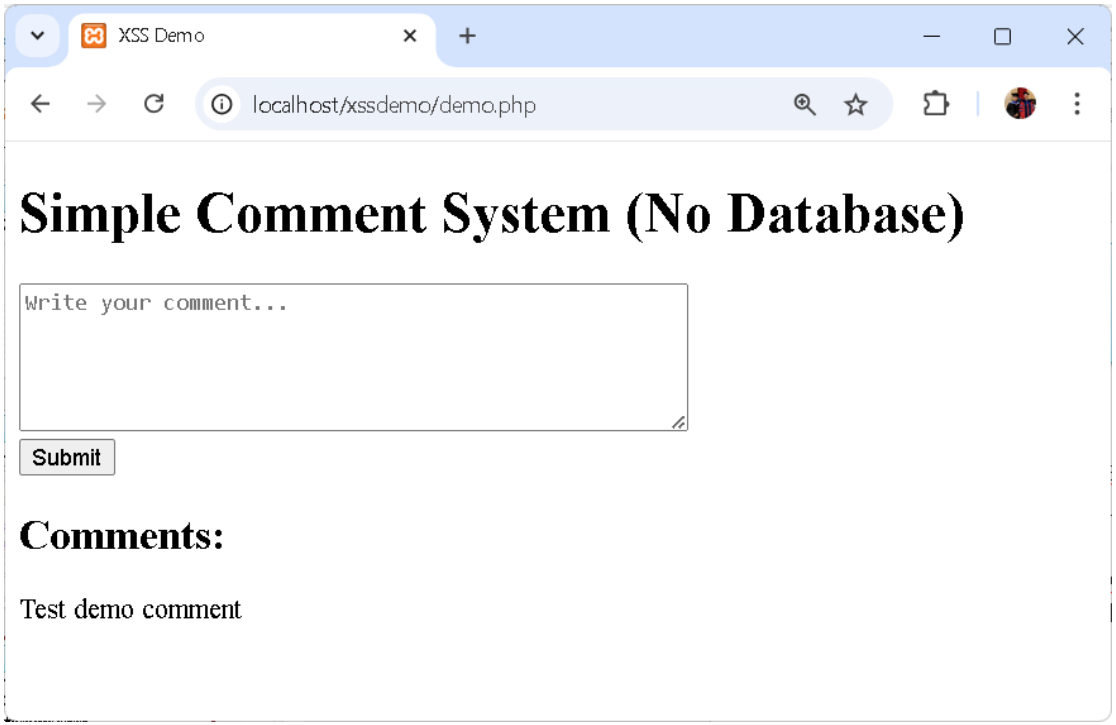


XSS Testing

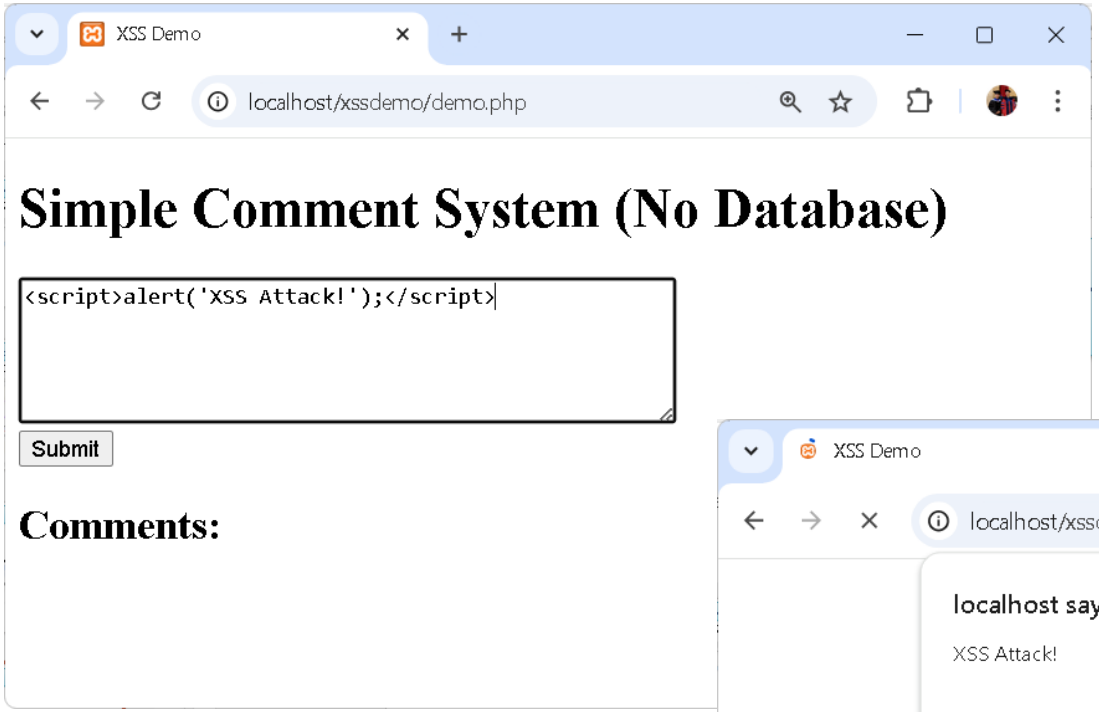
1) Buka aplikasi di browser, URL: **http://localhost/xssdemo/**

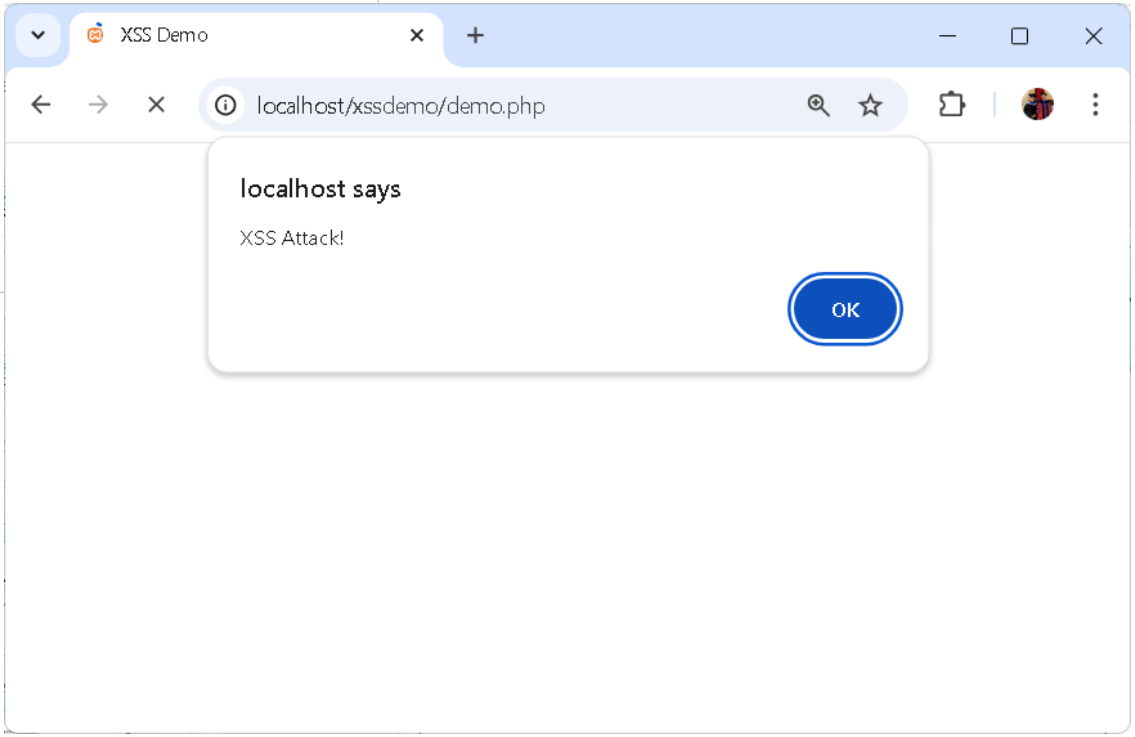
2) Masukkan komentar sederhana, seperti **Hello world!!!**, lalu klik tombol Submit. Komentar akan muncul di bawah input form.

3) Masukkan payload XSS berikut di textarea:
<script>alert('XSS Attack!');</script>



XSS Testing


XSS Demo
localhost/xssdemo/demo.php
Simple Comment System (No Database)
<script>alert('XSS Attack!');</script>
Submit
Comments:


XSS Demo
localhost/xssdemo/demo.php
localhost says
XSS Attack!
OK

Preventive Measures

Untuk mencegah serangan XSS, kita dapat menggunakan fungsi **htmlspecialchars()** untuk memfilter input dari user. Modifikasi file **demo.php** sebagai berikut:

Perbaikan untuk Pencegahan XSS:

```
// Proses form submission
if ($_SERVER['REQUEST_METHOD'] === 'POST' && !empty($_POST['comment'])) {
    // Filter input menggunakan htmlspecialchars
    $comments[] = htmlspecialchars($_POST['comment'], ENT_QUOTES, 'UTF-8');
}
```

Fungsi **htmlspecialchars()** akan melakukan konversi karakter spesial (seperti <, >, " dan ') menjadi entitas HTML sehingga tidak dieksekusi sebagai kode oleh browser, melainkan ditampilkan sebagai teks biasa.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T